Pontifícia Universidade Católica do Paraná

Engenharia de Software

Disciplina: Arquitetura e Soluções Cloud

Professor: Manoel Valerio da Silveira Neto

FuelWise

Documentação de Arquitetura de Software

arc42, C4 Model (Níveis 1 a 4), UML, Diagrama de Entidade-Relacionamento
e Software Architecture Canvas

TDE Grupo, Entrega 2: Documentação Software PJBL (Completo)

Integrantes

Artur Ribeiro de Souza
Cristhian Rezende
Lucas Andriani Theiss
Vinicius da Rosa Padilha

Curitiba
Outubro de 2026

Sumário

Software Architecture Canvas **4**

1. Introdução e Objetivos **5**
1.1 Visão Geral dos Requisitos . . . 5
1.2 Objetivos de Qualidade . . . 5
1.3 Partes Interessadas . . . 6

2. Restrições Arquiteturais **7**
2.1 Restrições Técnicas . . . 7
2.2 Restrições Organizacionais . . . 7

3. Contexto e Escopo **8**
3.1 Contexto Negocial . . . 8
3.2 Contexto Técnico . . . 9

4. Estratégia de Solução **10**
4.1 Implementação atual . . . 10
4.2 Arquitetura alvo com os estilos arquiteturais . . . 12

5. Visão de Blocos de Construção **16**
5.1 Nível 1: Visão Sistêmica Geral (caixa-branca do sistema) . . . 16
5.2 Nível 2: Decomposição da API (caixa-branca) . . . 16
5.3 Nível 3: Abastecimento e Pagamento . . . 18
5.4 Nível 3: Cadastro e Aprovação de Transportadora . . . 20
5.5 Estrutura de código de um slice . . . 21

6. Visão de Tempo de Execução **23**
6.1 UML de Classes: modelo de domínio (C4 Nível 4) . . . 23
6.2 UML de Classes: camadas da Clean Architecture e contratos SOLID (C4 Nível 4) . . . 23
6.3 UML de Componentes: um Vertical Slice (C4 Nível 4) . . . 26
6.4 Cenário: Abastecimento e Pagamento (RF15) . . . 27
6.5 Cenário: Aprovação de Cadastro de Transportadora (RF01, RF14) . . . 29
6.6 Cenário: Registro e Verificação de Denúncia (RF16) . . . 30
6.7 Cenário: Conciliação de Pagamento . . . 31

7. Visão de Implantação **32**
7.1 Nível de Infraestrutura 1: homologação (implementado) . . . 32
7.2 Nível de Infraestrutura 1: produção (proposta no Azure) . . . 33
7.3 Ambientes . . . 34

8. Conceitos Transversais **35**
8.1 Segurança . . . 35
8.2 Comunicação . . . 35
8.3 Persistência . . . 35
8.4 Notificações . . . 38
8.5 Tratamento de Erros e Resiliência . . . 38
8.6 Observabilidade . . . 39
8.7 Multi-tenancy . . . 39
8.8 Vertical Slice, SOLID e Clean Architecture . . . 39
8.9 Testes Unitários de Arquitetura . . . 40
8.10 Comunicação por eventos (proposta) . . . 42

9. Decisões Arquiteturais **44**

10. Requisitos de Qualidade **48**
10.1 Árvore de Qualidade 48
10.2 Cenários de Qualidade 49

11. Riscos e Débitos Técnicos **50**

12. Glossário **51**

Repositório do código **52**

Lista de Figuras

1 Software Architecture Canvas do FuelWise 4
2 Diagrama de Contexto do Sistema (C4 Nível 1) 8
3 Diagrama de Contêineres, implementação atual (C4 Nível 2) 11
4 Diagrama de Contêineres, arquitetura alvo (C4 Nível 2) 13
5 Diagrama de Componentes da API (C4 Nível 3) 17
6 Componentes do abastecimento e pagamento (C4 Nível 3) 19
7 Componentes do cadastro e aprovação (C4 Nível 3) 20
8 UML de Classes do domínio (C4 Nível 4) 23
9 UML de Classes do núcleo financeiro por camada (C4 Nível 4) 24
10 UML de Classes das integrações e do composition root (C4 Nível 4) 25
11 UML de Componentes do slice payments/pay (C4 Nível 4) 26
12 UML de Sequência: abastecimento e pagamento (C4 Nível 4) 28
13 UML de Sequência: aprovação de transportadora (C4 Nível 4) 29
14 UML de Sequência: denúncia (C4 Nível 4) 30
15 UML de Sequência: conciliação de pagamento (C4 Nível 4) 31
16 Implantação em homologação 32
17 Implantação proposta no Microsoft Azure 33
18 DER: cadastro e frota 36
19 DER: abastecimento, pagamento e denúncia 37
20 DER: acesso, notificações e auditoria 38

Software Architecture Canvas

Template: Software Architecture Canvas do arc42 (Patrick Roos), na versão Architecture Inception Canvas, apresentado em aula. Cada bloco corresponde a uma seção do arc42: Functional Overview e Quality Goals na Seção 1, Organisational e Technical Constraints na Seção 2, Business Context na Seção 3, Architectural Hypotheses na Seção 9 e Technical Challenges & Risks na Seção 11.

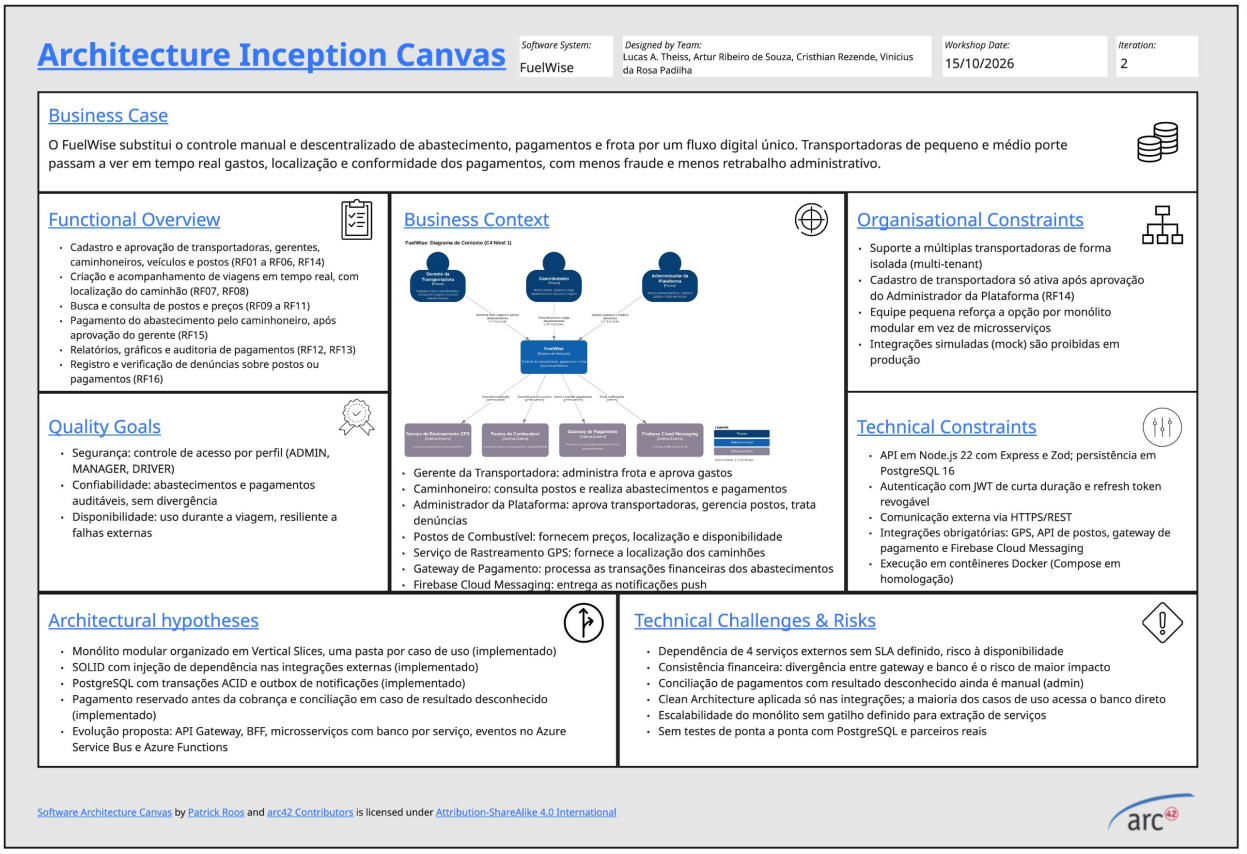


Figura 1: Software Architecture Canvas do FuelWise

1. Introdução e Objetivos

1.1 Visão Geral dos Requisitos

O FuelWise é uma plataforma web destinada a transportadoras de pequeno e médio porte, com foco na centralização do controle de combustível, pagamentos e gestão da frota.

O sistema busca substituir processos manuais e descentralizados por um fluxo integrado de cadastro, autorização, abastecimento, pagamento e acompanhamento da frota.

Os principais requisitos funcionais são listados abaixo, com o caso de uso (slice) que implementa cada um no backend:

Código	Requisito	Slice no código
RF01	Cadastrar transportadora na plataforma	`auth/register-carrier`
RF02	Cadastrar gerente da transportadora	`auth/register-carrier`
RF03	Cadastrar caminhoneiros	`drivers/create`
RF04	Cadastrar veículos da frota	`vehicles/create`
RF05	Cadastrar postos de combustível	`stations/create`
RF06	Manter tabela de preços dos combustíveis por posto	`stations/add-price`, `stations/prices`
RF07	Relacionar caminhoneiro a veículo, criando uma viagem	`trips/start`
RF08	Acompanhar em tempo real a localização do caminhão	`vehicles/location`
RF09	Localizar postos de combustível disponíveis	`stations/list`, `stations/external`
RF10	Visualizar dados dos postos	`stations/detail`
RF11	Acessar painel com postos filiados	`stations/list?isAffiliated=true`
RF12	Gerar relatórios e gráficos de abastecimento	`analytics/consumption`
RF13	Auditar dados de pagamentos das viagens	`analytics/audit`
RF14	Aprovar ou recusar solicitações de cadastro de transportadora	`carriers/list-pending`, `carriers/decide`
RF15	Realizar pagamento de abastecimento pelo caminhoneiro, após aprovação do gerente	`fuelings/create`, `fuelings/approve`, `payments/pay`
RF16	Registrar e verificar denúncias sobre postos ou pagamentos	`complaints/create`, `complaints/moderate`

1.2 Objetivos de Qualidade

Prioridade	Objetivo	Justificativa
Alta	Segurança	O sistema manipula dados de usuários, viagens e pagamentos e precisa controlar o acesso às operações por perfil e por transportadora.
Alta	Confiabilidade	Abastecimentos e pagamentos devem ser registrados corretamente para permitir auditoria (RF13). Uma cobrança não pode ser feita duas vezes.
Alta	Disponibilidade	O caminhoneiro precisa acessar o sistema durante a viagem, mesmo com falhas pontuais em serviços externos.
Média	Desempenho	Consultas de postos, localização e operações de abastecimento devem responder em tempo adequado ao uso operacional.
Média	Manutenibilidade	Cada caso de uso fica isolado em um slice, e as integrações externas podem ser trocadas sem alterar as regras de negócio.

1.2.1 Rastreabilidade entre requisitos e qualidade

Objetivo de qualidade	Requisitos funcionais relacionados
Segurança	RF01, RF02, RF03, RF14 (cadastro e aprovação controlados por perfil)
Confiabilidade	RF13, RF15, RF16 (pagamento, auditoria, denúncia)
Disponibilidade	RF08, RF09, RF15 (uso durante a viagem)
Desempenho	RF09, RF10, RF11 (consultas de postos)
Manutenibilidade	Todos, pela organização em Vertical Slices descrita na Seção 5

1.3 Partes Interessadas

Parte interessada	Expectativas
Gerente da Transportadora	Administrar frota e caminhoneiros, acompanhar a localização e controlar e aprovar gastos.
Caminhoneiro	Consultar postos, registrar abastecimentos e pagar com agilidade durante a viagem.
Administrador da Plataforma	Manter postos e transportadoras cadastrados e confiáveis, conciliar pagamentos pendentes e tratar denúncias (RF16).
Equipe de Desenvolvimento	Ter uma arquitetura organizada, testável e adequada à evolução do sistema.

2. Restrições Arquiteturais

2.1 Restrições Técnicas

- API desenvolvida em Node.js 22 (JavaScript ESM) com Express, validação de entrada com Zod e acesso ao banco pelo driver `pg`.
- Persistência utilizando PostgreSQL 16.
- Aplicação web prevista em React. O frontend ainda não foi implementado; a API já expõe todos os endpoints que ele vai usar.
- Autenticação com JWT de curta duração (15 minutos) e refresh token revogável, guardado como hash no banco.
- Comunicação entre aplicações e serviços externos utilizando HTTPS e APIs REST.
- Utilização de um serviço externo de rastreamento GPS para localização dos veículos.
- Utilização de um gateway externo para processamento dos pagamentos.
- Utilização de Firebase Cloud Messaging para envio de notificações.
- Execução em contêineres Docker, orquestrados por Docker Compose em homologação.
- Integrações externas simuladas (`INTEGRATION_MODE=mock`) em homologação. O próprio código impede o uso de mocks quando `NODE_ENV=production`.

2.2 Restrições Organizacionais

- O sistema deve suportar múltiplas transportadoras de forma isolada (dados de uma transportadora não podem ser acessados por outra).
- O cadastro de uma transportadora só se torna ativo após aprovação do Administrador da Plataforma (RF14), o que introduz um fluxo assíncrono de ativação.
- A equipe é pequena, o que reforça a necessidade de uma arquitetura simples de operar: um monólito modular nesta fase, com a evolução para os estilos da Seção 4.2 planejada para quando houver demanda.
- Os provedores comerciais de GPS, postos e pagamento ainda não foram contratados. Os adaptadores HTTP seguem contratos de referência que precisarão ser ajustados ao fornecedor escolhido.

3. Contexto e Escopo

3.1 Contexto Negocial

O FuelWise atua como elemento central entre os usuários da transportadora e os serviços externos relacionados ao abastecimento.

Elemento externo	Interação com o FuelWise
Gerente da Transportadora	Administra frota e caminhoneiros, aprova abastecimentos.
Caminhoneiro	Consulta postos, registra e paga abastecimentos.
Administrador da Plataforma	Aprova transportadoras, gerencia postos, concilia pagamentos e trata denúncias.
Postos de Combustível	Disponibilizam informações de localização, preços e disponibilidade.
Serviço de Rastreamento GPS	Fornece a localização dos caminhões.
Gateway de Pagamento	Processa e confirma os pagamentos dos abastecimentos.
Firebase Cloud Messaging	Entrega as notificações push.

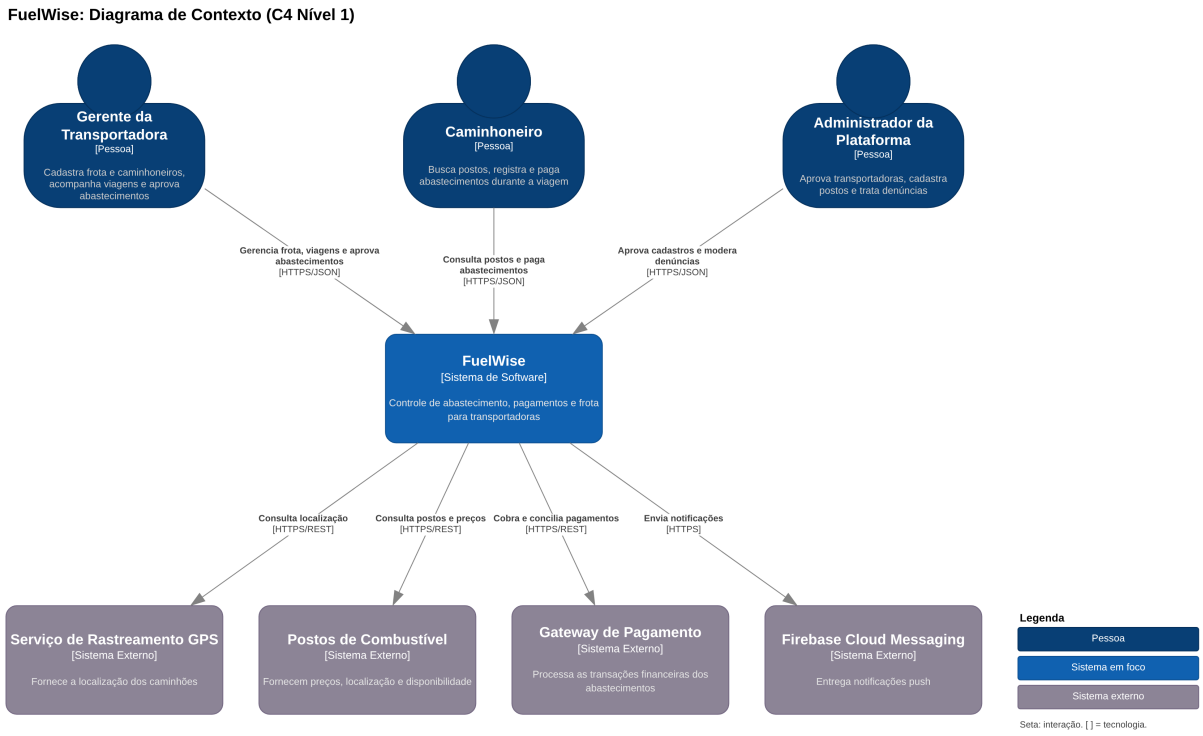


Figura 2: Diagrama de Contexto do Sistema (C4 Nível 1)

3.2 Contexto Técnico

As principais integrações técnicas são realizadas por APIs utilizando HTTPS.

Canal	Protocolo	Observação
Aplicação web para a API	HTTPS/JSON, `Authorization: Bearer <JWT>`	Base `/api/v1`. Rate limit nas rotas de login, refresh e cadastro.
API para serviço GPS	HTTPS/REST	`GET /vehicles/{id}/location`. Em falha, a API devolve a última posição salva.
API para API de postos	HTTPS/REST	`GET /stations`. Postos externos são só consultados, não gravados.
API para gateway de pagamento	HTTPS/REST com chave de idempotência	`POST /charges` e `GET /charges/{id}` para conciliação.
API para Firebase	FCM HTTP v1	Enviado pelo worker do outbox, nunca dentro da requisição do usuário.
API para PostgreSQL	Protocolo do PostgreSQL (driver `pg`)	Somente a API acessa o banco.

4. Estratégia de Solução

4.1 Implementação atual

A arquitetura do FuelWise está implementada como um **monólito modular**: uma API Node.js, um banco PostgreSQL e os serviços externos.

Dentro da API, o código é organizado em **Vertical Slices**: cada caso de uso tem a sua pasta em `src/slices/<domínio>/<ação>/`, com um `route.js` (rota HTTP, perfil e validação) e um `useCase.js` (regra do caso de uso). São 34 slices em 11 domínios. A infraestrutura compartilhada fica em `src/shared`.

O núcleo financeiro (registro, aprovação, pagamento e conciliação de abastecimentos) segue a **Clean Architecture**. As regras de negócio ficam em `src/domain` (cálculo do valor, transições de estado, idempotência), sem depender de Express, banco ou HTTP. Os casos de uso desse núcleo importam só o domínio e recebem o acesso a dados e aos serviços externos como portas.

Os arquivos `src/app.js` e `src/server.js` são o *composition root*: criam uma vez os repositórios e os adaptadores de pagamento, GPS, postos e notificação e injetam esses objetos nos slices e no worker. Assim os casos de uso recebem portas pequenas (princípios DIP e ISP do SOLID) e podem usar a implementação real ou o mock sem mudar o código.

Para pagamentos, a estratégia prioriza confiabilidade: o pagamento é gravado como `PROCESSING` antes de chamar o gateway, existe no máximo um pagamento por abastecimento, e uma resposta ausente vira `UNKNOWN` para conciliação, nunca uma nova cobrança. Notificações são gravadas numa tabela de outbox na mesma transação e enviadas depois por um worker.

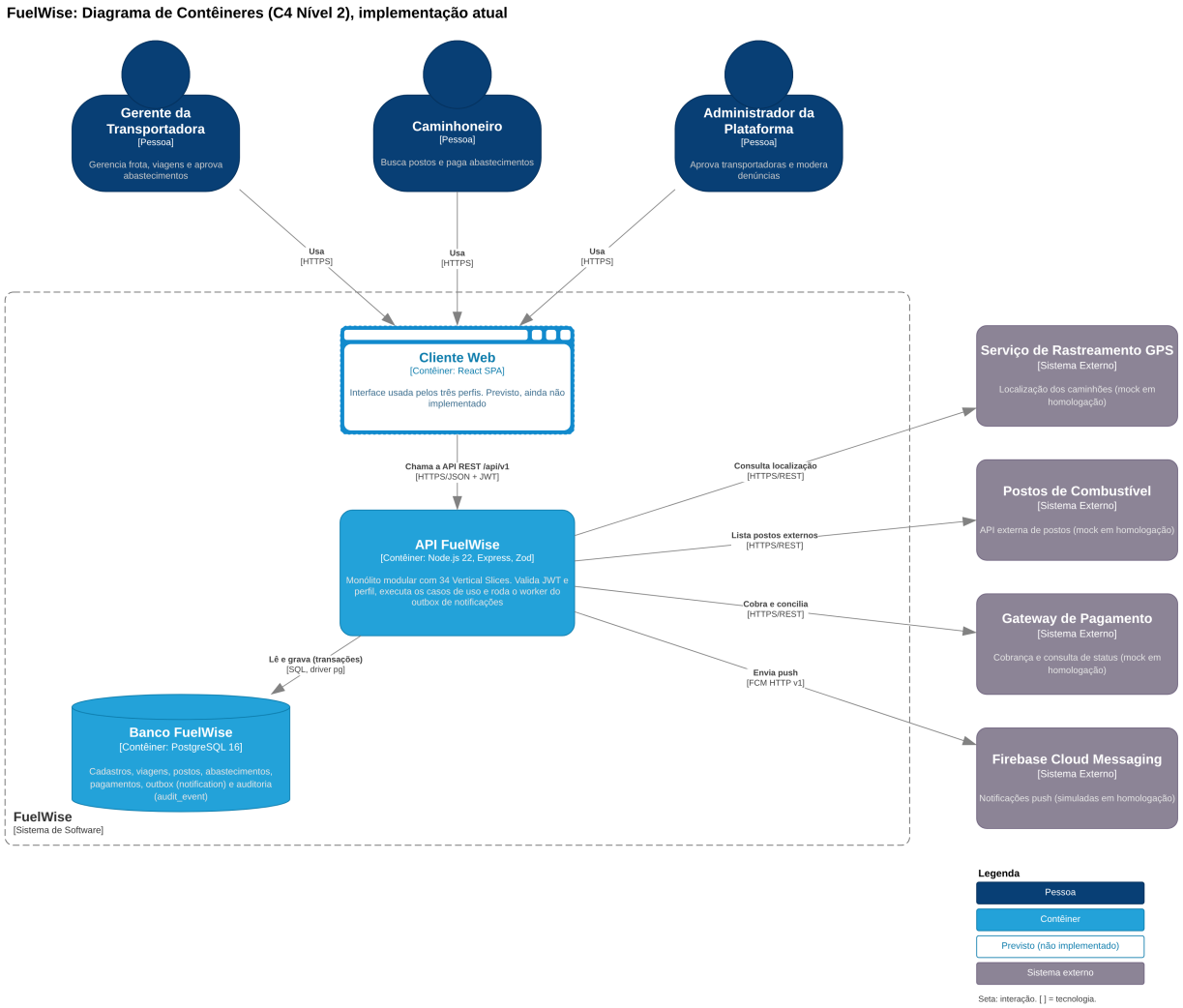


Figura 3: Diagrama de Contêineres, implementação atual (C4 Nível 2)

Relações do diagrama de contêineres atual:

Origem	Destino	Interação [tecnologia]
Gerente, Caminhoneiro e Administrador	Cliente Web (previsto)	Usa [HTTPS]
Cliente Web	API FuelWise	Chama a API REST /api/v1 [HTTPS/JSON + JWT]
API FuelWise	Banco FuelWise	Lê e grava em transações [SQL, driver pg]
API FuelWise	Serviço de Rastreamento GPS	Consulta localização [HTTPS/REST]
API FuelWise	Postos de Combustível	Lista postos externos [HTTPS/REST]
API FuelWise	Gateway de Pagamento	Cobra e concilia [HTTPS/REST]
API FuelWise	Firebase Cloud Messaging	Envia push [FCM HTTP v1]

4.2 Arquitetura alvo com os estilos arquiteturais

Quando o volume de transportadoras crescer, a proposta é evoluir o monólito para os estilos apresentados na disciplina. Cada domínio de slices já é uma fronteira natural de serviço, então a extração não exige reescrever as regras.

Estilo	Situação	Como aparece no FuelWise
Vertical Slice	Implementado	34 slices em `src/slices`, um por caso de uso.
SOLID e Clean Architecture	Implementado no núcleo financeiro	Camada de domínio em `src/domain`; abastecimento e pagamento usam só domínio e portas; integrações (pagamento, GPS, postos e notificação) injetadas pelo composition root. Detalhes na Seção 8.8.
Testes unitários de arquitetura	Implementado	`tests/architecture-rules.test.js`, executado no CI (Seção 8.9).
Microsserviços + Database per Service	Proposto	Quatro serviços (Cadastro, Viagem, Postos, Pagamento), cada um com seu banco.
API Gateway	Proposto	Azure API Management como entrada única, validando JWT e aplicando rate limit.
Backend for Frontend	Proposto	BFF Portal para gerente e administrador, BFF App para o caminhoneiro.
Microfrontend	Proposto	Portal Web com um shell e os módulos frota, financeiro e admin.
Arquitetura orientada a eventos (EDA)	Proposto	Tópico `fuelwise-eventos` no Azure Service Bus. O outbox atual vira o publicador.
Serverless	Proposto	Azure Functions para notificações e para a conciliação diária de pagamentos `UNKNOWN`.

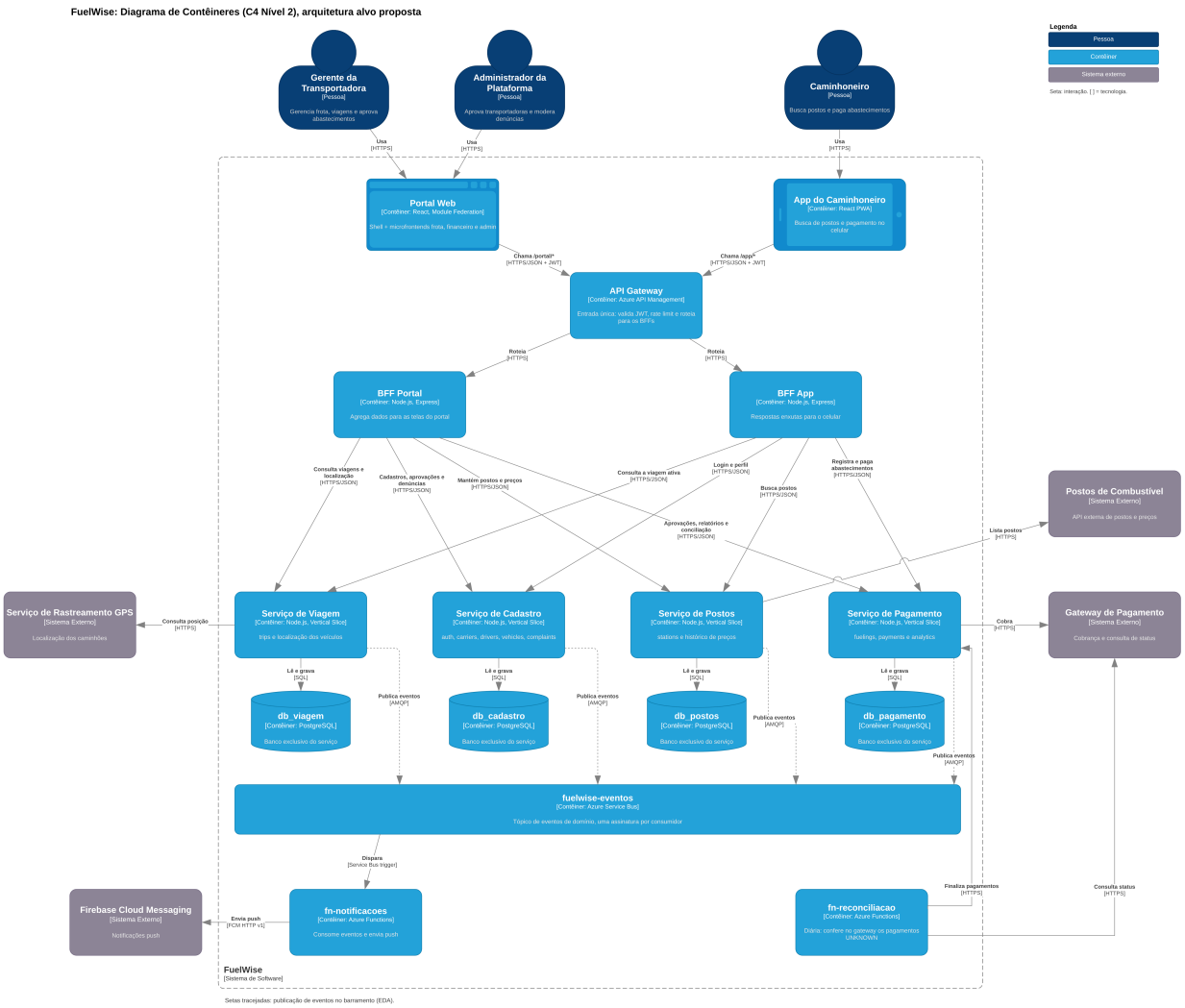


Figura 4: Diagrama de Contêineres, arquitetura alvo (C4 Nível 2)

Relações do diagrama de contêineres alvo:

Origem	Destino	Interação [tecnologia]
Gerente e Administrador	Portal Web	Usa [HTTPS]
Caminhoneiro	App do Caminhoneiro	Usa [HTTPS]
Portal Web	API Gateway	Chama /portal/* [HTTPS/JSON + JWT]
App do Caminhoneiro	API Gateway	Chama /app/* [HTTPS/JSON + JWT]
API Gateway	BFF Portal e BFF App	Roteia [HTTPS]
BFF Portal	Serviço de Viagem	Consulta viagens e localização [HTTPS/JSON]
BFF Portal	Serviço de Cadastro	Cadastros, aprovações e denúncias [HTTPS/JSON]
BFF Portal	Serviço de Postos	Mantém postos e preços [HTTPS/JSON]
BFF Portal	Serviço de Pagamento	Aprovações, relatórios e conciliação [HTTPS/JSON]
BFF App	Serviço de Viagem	Consulta a viagem ativa [HTTPS/JSON]
BFF App	Serviço de Cadastro	Login e perfil [HTTPS/JSON]
BFF App	Serviço de Postos	Busca postos [HTTPS/JSON]
BFF App	Serviço de Pagamento	Registra e paga abastecimentos [HTTPS/JSON]
Cada serviço	O próprio banco (`db_viagem`, `db_cadastro`, `db_postos`, `db_pagamento`)	Lê e grava [SQL]
Cada serviço	`fuelwise-eventos` (Azure Service Bus)	Publica eventos [AMQP]
`fuelwise-eventos`	`fn-notificacoes`	Dispara [Service Bus trigger]
`fn-notificacoes`	Firebase Cloud Messaging	Envia push [FCM HTTP v1]
`fn-reconciliacao`	Gateway de Pagamento	Consulta status [HTTPS]
`fn-reconciliacao`	Serviço de Pagamento	Finaliza pagamentos [HTTPS]
Serviço de Viagem	Serviço de Rastreamento GPS	Consulta posição [HTTPS]
Serviço de Postos	Postos de Combustível	Lista postos [HTTPS]
Serviço de Pagamento	Gateway de Pagamento	Cobra [HTTPS]

A divisão em quatro serviços segue os domínios dos slices:

Serviço proposto	Domínios de slices atuais
Serviço de Cadastro	`auth`, `carriers`, `drivers`, `vehicles`, `complaints`
Serviço de Viagem	`trips`, `vehicles/location`
Serviço de Postos	`stations`
Serviço de Pagamento	`fuelings`, `payments`, `analytics`

No frontend, o Portal Web é dividido por área de negócio. O shell carrega os outros módulos em tempo de execução com Module Federation, e todos falam com o BFF Portal. O app do caminhoneiro é uma aplicação única que fala com o BFF App.

Módulo do Portal Web	Telas	Requisitos
shell	Login, menu, sessão e roteamento entre os módulos	Acesso dos três perfis
frota	Caminhoneiros, veículos, viagens e localização	RF03, RF04, RF07, RF08
financeiro	Aprovação de abastecimentos, relatórios e auditoria de pagamentos	RF12, RF13, RF15
admin	Transportadoras pendentes, postos e preços, denúncias e conciliação	RF05, RF06, RF14, RF16

5. Visão de Blocos de Construção

5.1 Nível 1: Visão Sistêmica Geral (caixa-branca do sistema)

Bloco	Responsabilidade
Aplicação Web (prevista)	Interface utilizada pelos usuários do FuelWise.
API FuelWise	Processar as operações e regras de negócio, autenticar e autorizar os usuários.
Banco de Dados	Armazenar usuários, transportadoras, veículos, viagens, postos, abastecimentos, pagamentos, denúncias, notificações e auditoria.
Worker de notificações	Ler a tabela de outbox e enviar as notificações. Roda no mesmo processo da API.
Integração com Postos	Consultar postos e preços externos.
Integração GPS	Obter a localização dos caminhões.
Integração de Pagamento	Cobrar e consultar o status das cobranças no gateway.
Integração Firebase	Enviar notificações push.

5.2 Nível 2: Decomposição da API (caixa-branca)

A API é decomposta em slices agrupados por domínio:

Domínio (`src/slices`)	Slices	Responsabilidade	Requisitos
`auth`	login, logout, me, refresh, register-carrier	Acesso, tokens e solicitação de cadastro	RF01, RF02
`carriers`	list-pending, decide	Fila e decisão de aprovação	RF14
`drivers`	create, list	Caminhoneiros da transportadora	RF03
`vehicles`	create, list, location	Frota e posição GPS	RF04, RF08
`trips`	start, list, finish	Viagens (motorista e veículo)	RF07
`stations`	create, list, detail, prices, add-price, external	Postos e histórico de preços	RF05, RF06, RF09 a RF11
`fuelings`	create, list, approve	Abastecimentos sujeitos à aprovação	RF15
`payments`	pay, get, reconcile	Cobrança, consulta e conciliação	RF15
`analytics`	audit, consumption	Auditoria e relatório de consumo	RF12, RF13
`complaints`	create, list, moderate	Denúncias e moderação	RF16
`notifications`	list, register-token	Caixa de notificações e token FCM	Transversal

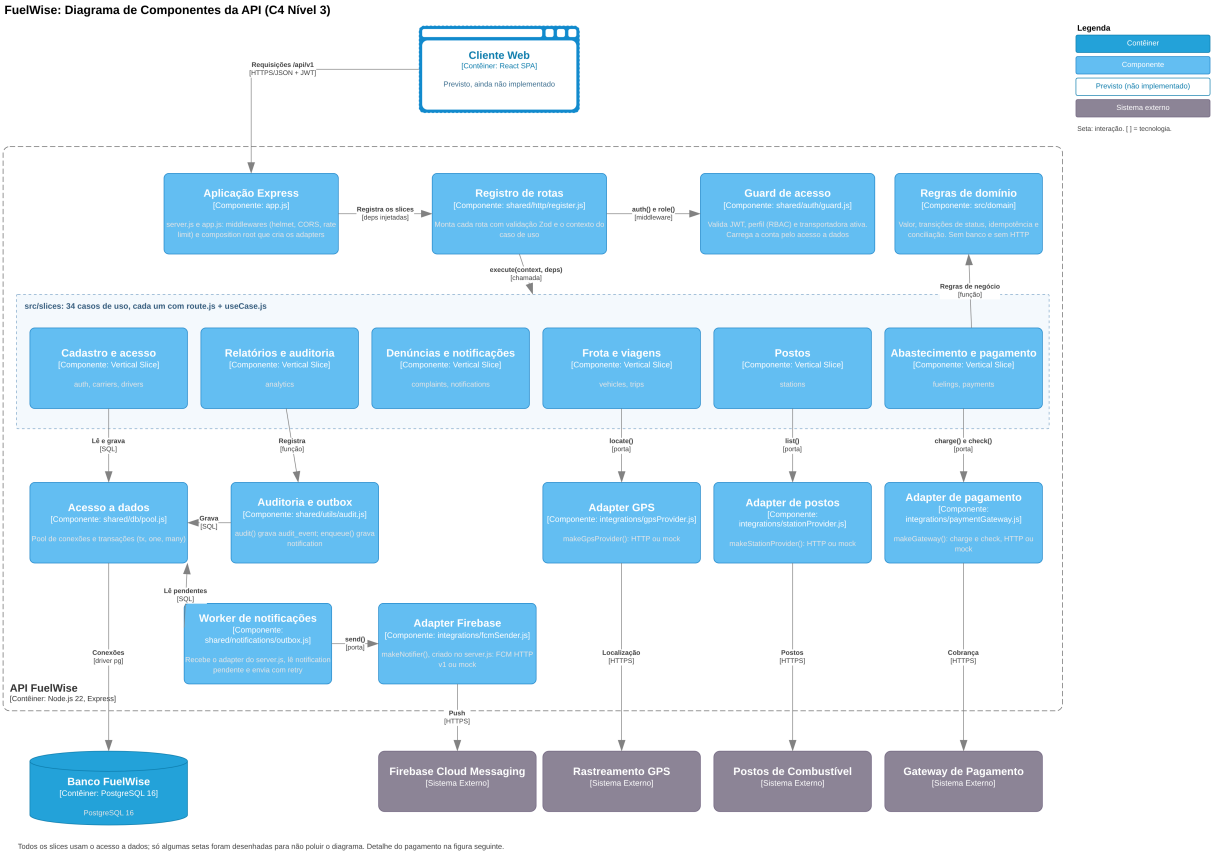


Figura 5: Diagrama de Componentes da API (C4 Nível 3)

5.3 Nível 3: Abastecimento e Pagamento

Componente	Arquivo	Responsabilidade
fuelings/create	`slices/fuelings/create`	O caminhoneiro registra o abastecimento. O valor é calculado pela regra de domínio com o preço vigente do posto, nunca enviado pelo cliente.
fuelings/approve	`slices/fuelings/approve`	O gerente aprova ou recusa. A regra de domínio valida a transição de estado.
Fueling store	`slices/fuelings/` `create` e `approve`, `repository.js`	Implementa `request()` e `decide()` em transações no PostgreSQL.
Rota de pagamento	`slices/payments/pay/` `route.js`	`POST /fuelings/:id/payment`, perfil DRIVER, validação do id.
Caso de uso de pagamento	`slices/payments/pay/` `useCase.js`	Reserva, cobra e finaliza usando apenas as portas `paymentStore` e `gateway`.
Payment store	`slices/payments/pay/` `repository.js`	Implementa `reserve()`, `finalize()` e `find()` no PostgreSQL.
Regras de domínio	`domain/` `fueling.js` e `payment.js`	Valor, transições de estado, chave de idempotência e quando um pagamento pode ser finalizado.
Adapter de pagamento	`shared/integrations/` `paymentGateway.js`	Implementa `charge()` e `check()` por HTTP ou mock.
payments/reconcile	`slices/payments/reconcile`	O administrador consulta o gateway e finaliza pagamentos pendentes.

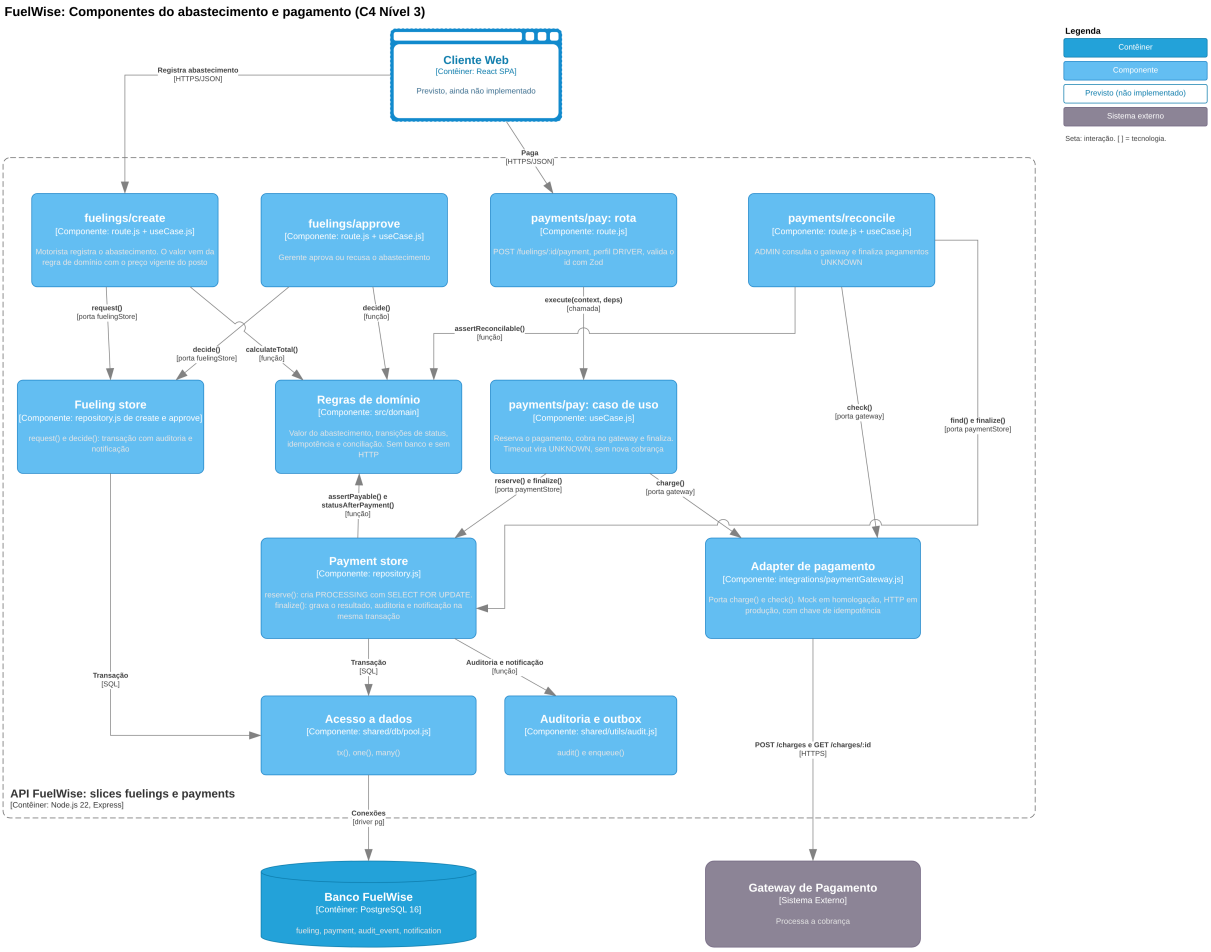


Figura 6: Componentes do abastecimento e pagamento (C4 Nível 3)

5.4 Nível 3: Cadastro e Aprovação de Transportadora

Módulo relevante por concentrar o fluxo de ativação de novos clientes na plataforma (RF01, RF14).

Componente	Responsabilidade
auth/register-carrier	Rota pública. Valida o CNPJ e grava a transportadora como `PENDING` junto com a conta do gerente.
auth/login e refresh	Emitem o JWT e o refresh token. Gerente de transportadora não ativa não consegue entrar.
carriers/list-pending	Lista as solicitações pendentes para o administrador.
carriers/decide	Aplica a decisão (`ACTIVE` ou `REJECTED` com motivo), registra auditoria e notifica o gerente.
Guard de acesso	Verifica token, perfil e se a transportadora está ativa em toda requisição autenticada.

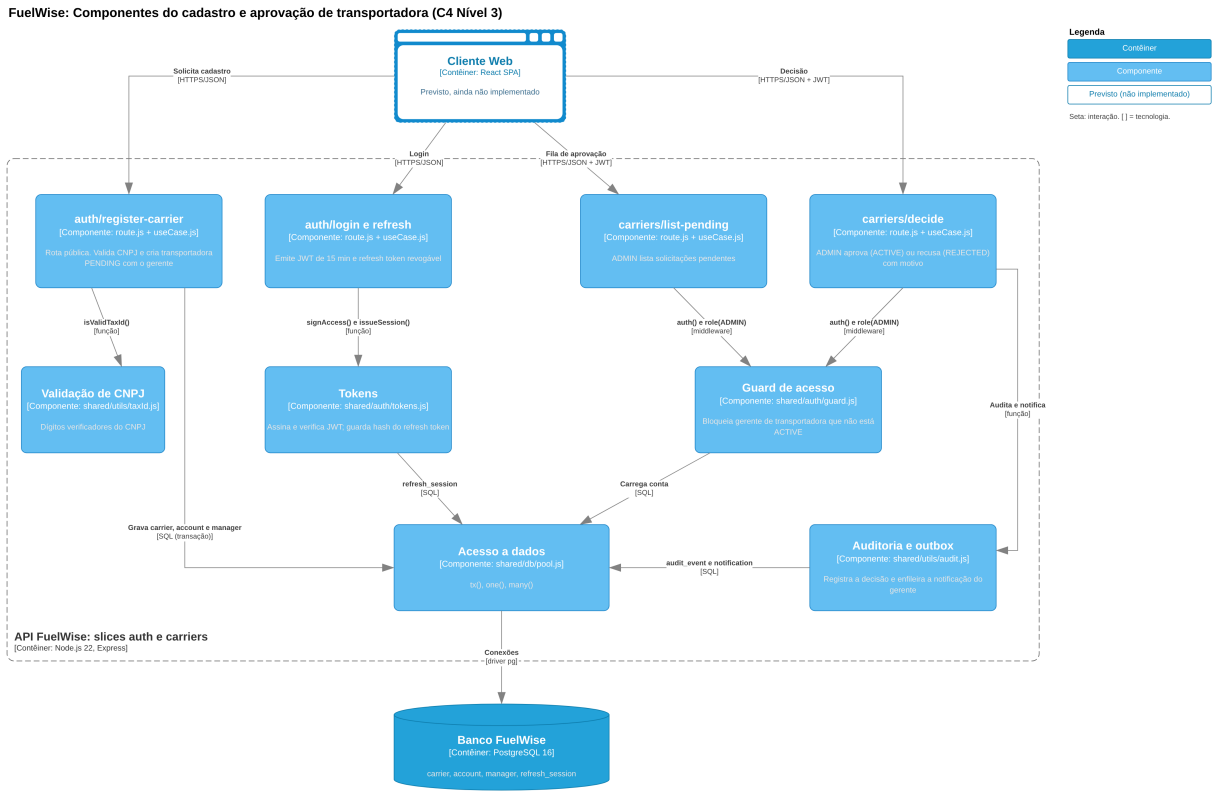


Figura 7: Componentes do cadastro e aprovação (C4 Nível 3)

5.5 Estrutura de código de um slice

Cada slice tem o mesmo formato. Abaixo, a estrutura de pastas e o slice de pagamento.

Código 1: Estrutura de pastas do código (resumo)

```
src/
  app.js                 composition root (cria e injeta repositórios e adapters)
  server.js              inicia a API e o worker do outbox (injeta o notifier)
  domain/                errors.js, fueling.js, payment.js (regras de negócio)
  shared/
    config.js            variáveis de ambiente e assertConfig()
    auth/                guard.js, tokens.js
    db/                  pool.js
    http/                register.js, errors.js, validate.js
    integrations/        paymentGateway.js, gpsProvider.js, stationProvider.js,
                         fcmSender.js, mockAdapters.js, httpClient.js
    notifications/       outbox.js
    utils/               audit.js, taxId.js
  slices/
    fuelings/
      create/    route.js, useCase.js, repository.js
      approve/   route.js, useCase.js, repository.js
      list/      route.js, useCase.js
    payments/
      pay/       route.js, useCase.js, repository.js
      get/       route.js, useCase.js
      reconcile/ route.js, useCase.js
    ... (mais 9 domínios)
```

Código 2: `src/slices/payments/pay/route.js`

```
export const register=(app,deps)=>route(app,'post','/api/v1/fuelings/:id/payment',{
  roles:['DRIVER'],params:z.object({id:uuid}),execute
},deps);
```

A rota declara o método, o caminho, o perfil permitido (`DRIVER`) e a validação do parâmetro. A regra fica no `useCase.js`, mostrado a seguir.

Código 3: `src/slices/payments/pay/useCase.js` (log de erro omitido)

```
export async function execute({actor,params},deps) {
  const {payment,newlyCreated}=await deps.paymentStore.reserve(actor,params.id);
  if (!newlyCreated) return {payment:payment,reused:true,note:'Não houve nova cobrança'};
  try {
    const gatewayResult=await deps.gateway.charge({paymentId:payment.id,
      amount:payment.amount,idempotencyKey:payment.idempotency_key});
    const saved=await deps.paymentStore.finalize(payment.id,gatewayResult,actor);
    return {payment:saved,reused:false,simulated:deps.isPaymentMock};
  } catch(error) {
    // No reply from gateway is NOT proof of rejection. Reconcile, do not charge again.
    const saved=await deps.paymentStore.finalize(payment.id,
      {status:'UNKNOWN',transactionId:null,message:'Falha de comunicação; requer
↪  conciliação'},actor);
    return {payment:saved,reused:false,requiresReconciliation:true,
      message:'Resultado desconhecido; não efetue nova cobrança'};
  }
}
```

O caso de uso não importa o driver do banco nem o cliente HTTP. Tudo o que ele usa chega pelo parâmetro `deps`. As regras de negócio do pagamento (chave de idempotência, abastecimento pagável, estado depois do resultado) ficam em `src/domain` e são aplicadas pelo `repository.js` dentro da transação.

Código 4: `src/slices/fuelings/approve/useCase.js`

```
import { decide } from '../../../domain/fueling.js';
export async function execute({actor,params,body},deps) {
  const fueling=await deps.fuelingStore.decide(actor,params.id,
    current=>decide(current,body.decision));
  return {fueling};
}
```

O repositório bloqueia a linha do abastecimento, entrega o registro atual para a regra `decide()` do domínio e grava o novo estado na mesma transação. Se o abastecimento já foi analisado, o domínio lança `DomainError` do tipo `conflict` e a API responde 409.

6. Visão de Tempo de Execução

Seguindo o mapeamento entre arc42 e C4 apresentado em aula, o Nível 4 do C4 (código) fica nesta seção: UML de Classes e de Componentes (6.1 a 6.3) e UML de Sequência dos cenários principais (6.4 a 6.7).

6.1 UML de Classes: modelo de domínio (C4 Nível 4)

As classes representam as tabelas de `database/migrations/001_initial.sql`. O backend é JavaScript e não declara classes de entidade; o diagrama mostra os registros e as associações que o banco garante.

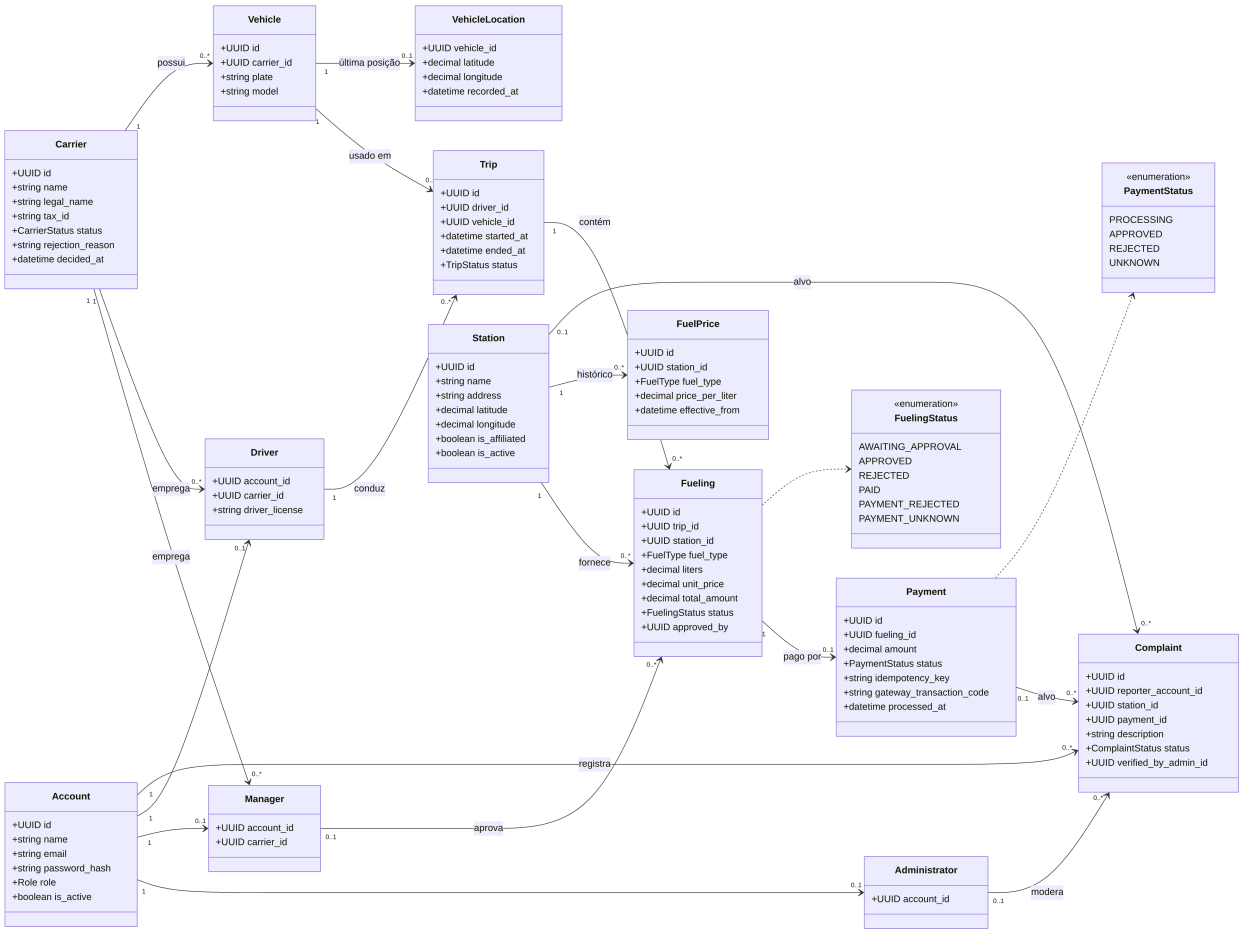


Figura 8: UML de Classes do domínio (C4 Nível 4)

6.2 UML de Classes: camadas da Clean Architecture e contratos SOLID (C4 Nível 4)

Os dois diagramas agrupam as classes pelas camadas da Clean Architecture. Os estereótipos `<<interface>>` representam as portas que os casos de uso recebem por injeção. Em JavaScript elas não são declaradas com `interface`; são objetos com os métodos mostrados, e cada implementação (repositório, adaptador HTTP ou mock) tem as mesmas assinaturas. Seta tracejada aberta é dependência; seta tracejada com triângulo vazado é realização da interface.

O primeiro diagrama mostra o núcleo financeiro. As dependências apontam para dentro: os casos de uso dependem das portas e das regras de domínio (Entities), e os repositórios PostgreSQL (Interface Adapters) implementam as portas e aplicam as regras do domínio dentro da transação. Nenhuma classe de `src/domain` depende de outra camada.

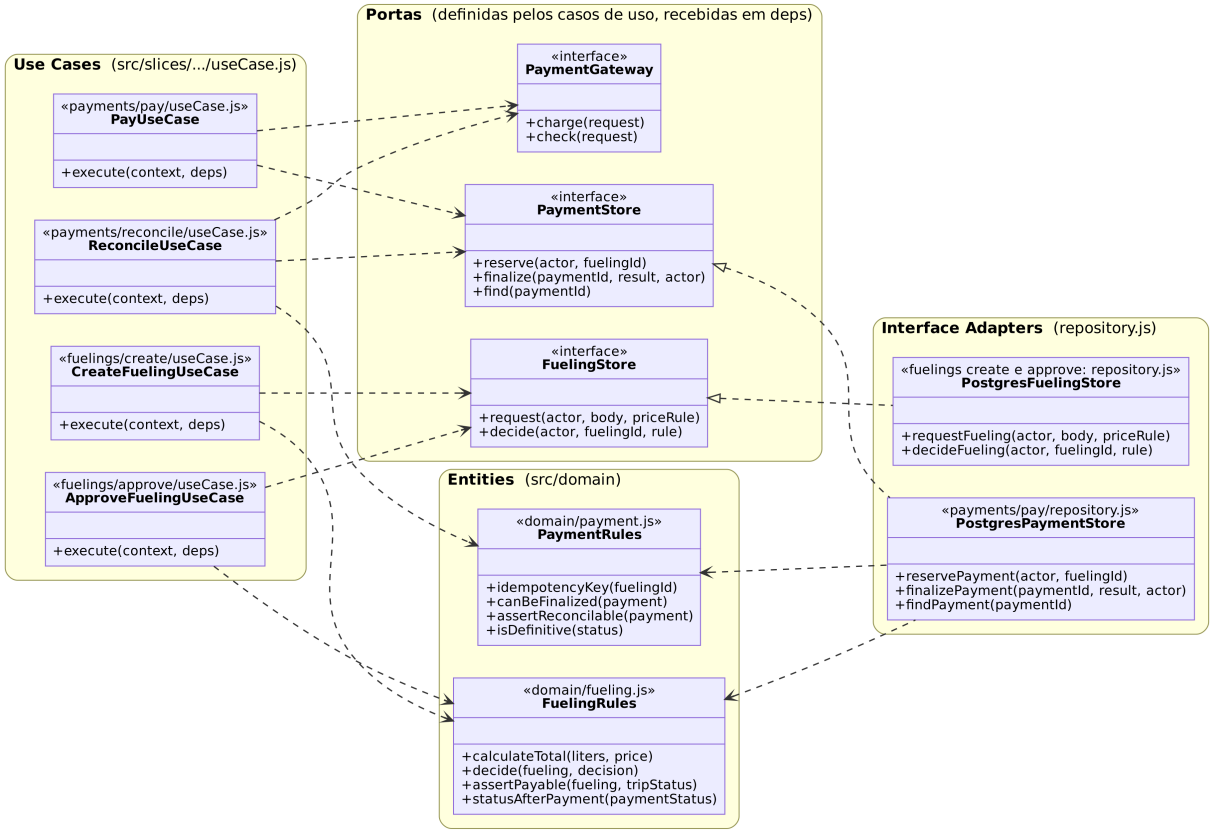


Figura 9: UML de Classes do núcleo financeiro por camada (C4 Nível 4)

O segundo diagrama mostra as integrações externas. O composition root (`app.js` e `server.js`) cria os adaptadores e os injeta; o caso de uso de localização e o worker do outbox só conhecem as portas.

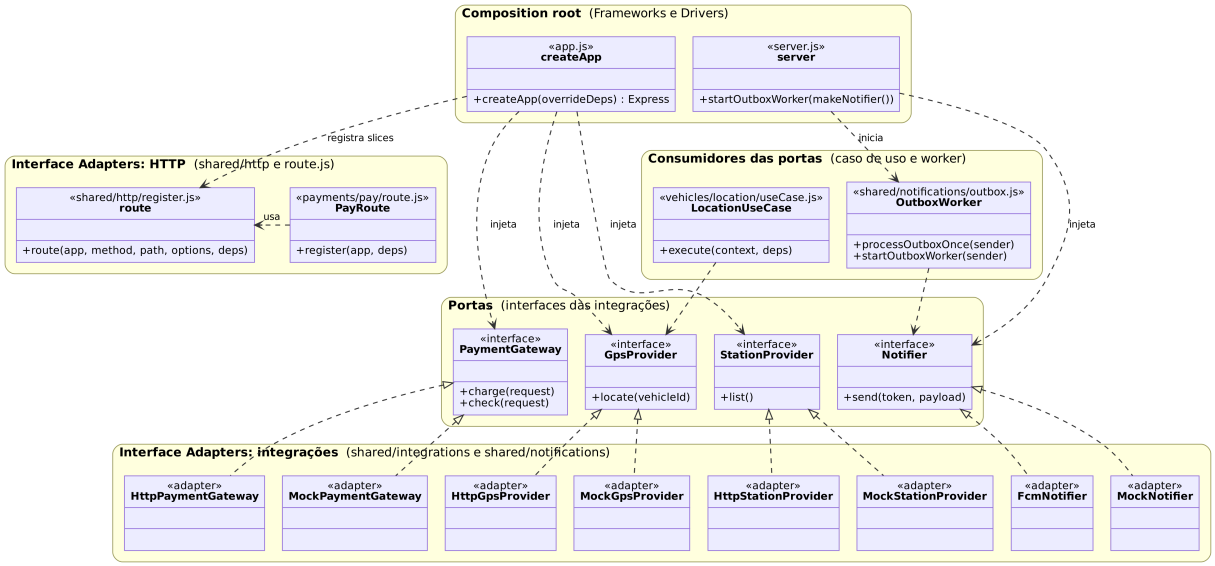


Figura 10: UML de Classes das integrações e do composition root (C4 Nível 4)

6.3 UML de Componentes: um Vertical Slice (C4 Nível 4)

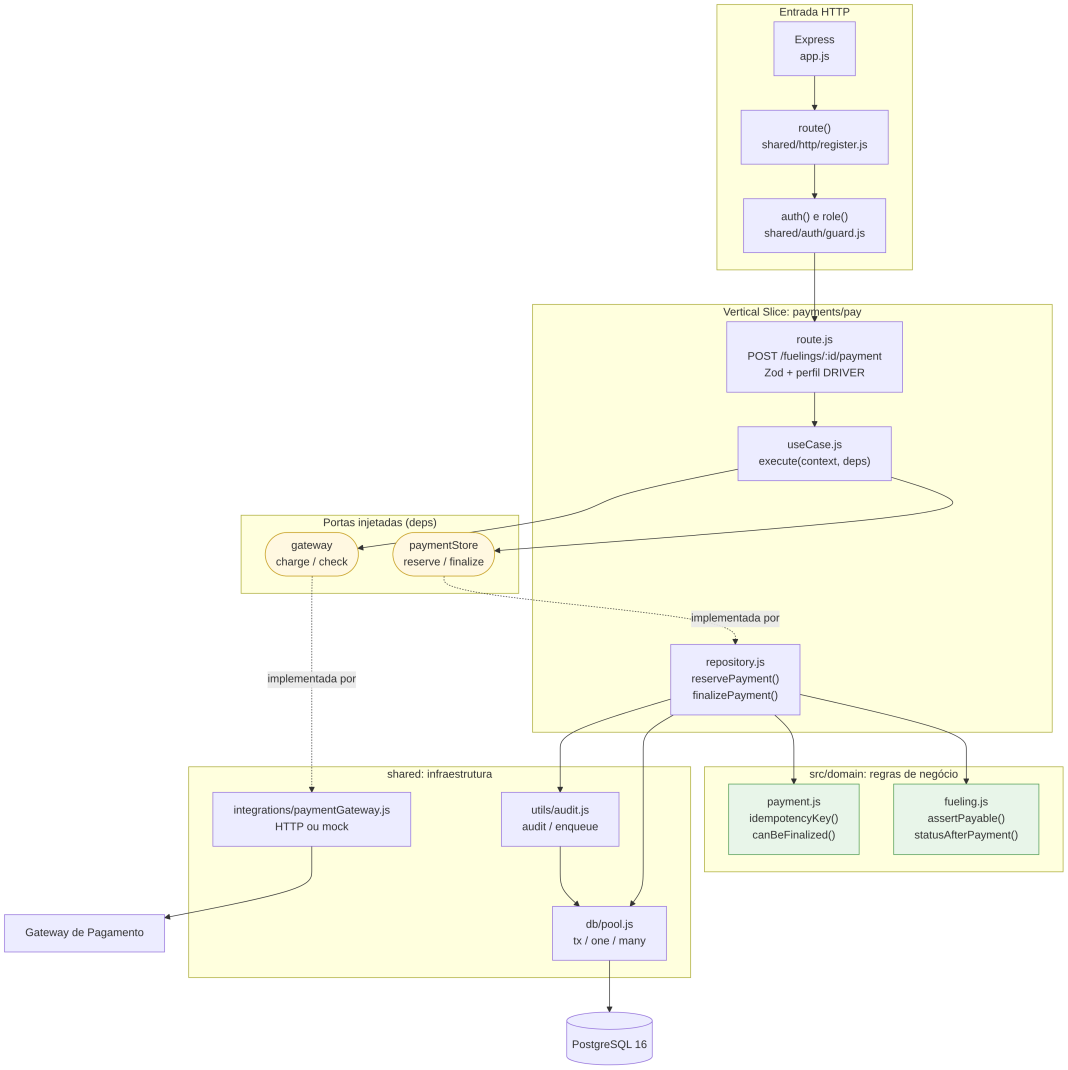


Figura 11: UML de Componentes do slice payments/pay (C4 Nível 4)

Porta	Fornecida por	Consumida por	Descrição
`fuelingStore` (request, decide)	slices fuelings/create e fuelings/approve, `repository.js`	slices fuelings/create e fuelings/approve	Persistência do abastecimento em transação
`paymentStore` (reserve, finalize, find)	slice payments/pay, `repository.js`	slices payments/pay e payments/reconcile	Persistência do pagamento em transação
`gateway` (charge, check)	`paymentGateway.js` (HTTP ou mock)	slices payments/pay e payments/reconcile	Abstrai o protocolo do gateway externo
`gps` (locate)	`gpsProvider.js`	slice vehicles/location	Localização do veículo
`stations` (list)	`stationProvider.js`	slice stations/external	Postos externos
`notifier` (send)	`fcmSender.js`, injetado por `server.js`	worker `notifications/outbox.js`	Canal de notificação (Firebase ou mock)

Os adaptadores ficam em `src/shared/integrations/`. As portas `fuelingStore` e `paymentStore` são montadas em `src/app.js`; o `notifier` é criado em `src/server.js` e entregue ao worker.

6.4 Cenário: Abastecimento e Pagamento (RF15)

1. O caminhoneiro registra o abastecimento informando viagem, posto, combustível e litros.
2. A API confere se a viagem está ativa e pertence ao caminhoneiro e busca o preço vigente do posto. A regra de domínio `calculateTotal()` calcula o valor, e o abastecimento é gravado como `AWAITING_APPROVAL`. Os gerentes da transportadora recebem uma notificação.
3. O gerente aprova o abastecimento. A regra `decide()` só aceita abastecimentos que ainda aguardam aprovação.
4. O caminhoneiro solicita o pagamento.
5. O caso de uso reserva o pagamento: bloqueia a linha do abastecimento e, se já existir pagamento, devolve o existente sem nova cobrança. Senão, a regra `assertPayable()` confere se o abastecimento está aprovado e a viagem ativa, e o pagamento é criado como `PROCESSING`.
6. O gateway é chamado com a chave de idempotência `fuelwise:<fuelingId>`, gerada por `idempotencyKey()` no domínio.
7. Com resposta, o pagamento vira `APPROVED` ou `REJECTED`, e o abastecimento vira `PAID` ou `PAYMENT_REJECTED` (regra `statusAfterPayment()`). Sem resposta, o pagamento vira `UNKNOWN` e o abastecimento `PAYMENT_UNKNOWN`.
8. Auditoria e notificação são gravadas na mesma transação do resultado.

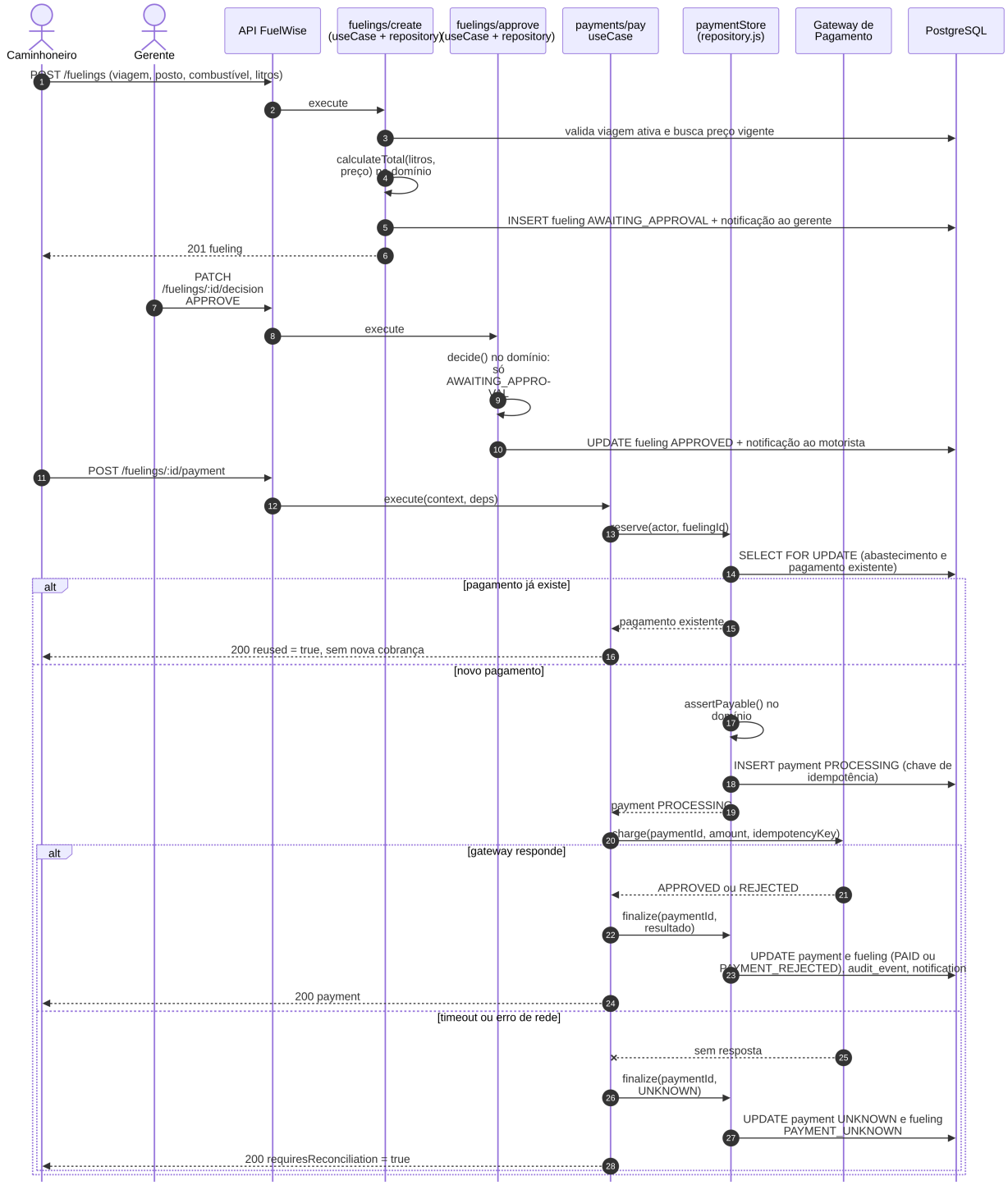


Figura 12: UML de Sequência: abastecimento e pagamento (C4 Nível 4)

6.5 Cenário: Aprovação de Cadastro de Transportadora (RF01, RF14)

1. O gerente envia o cadastro da transportadora pela aplicação web.
2. A API valida o CNPJ e grava, em uma transação, a transportadora `PENDING`, a conta e o gerente.
3. Enquanto a transportadora não está ativa, o login do gerente é recusado.
4. O administrador consulta a fila de pendentes.
5. O administrador aprova (`ACTIVE`) ou recusa (`REJECTED`, com motivo obrigatório).
6. A decisão gera auditoria e uma notificação para o gerente, enviada pelo worker do outbox.

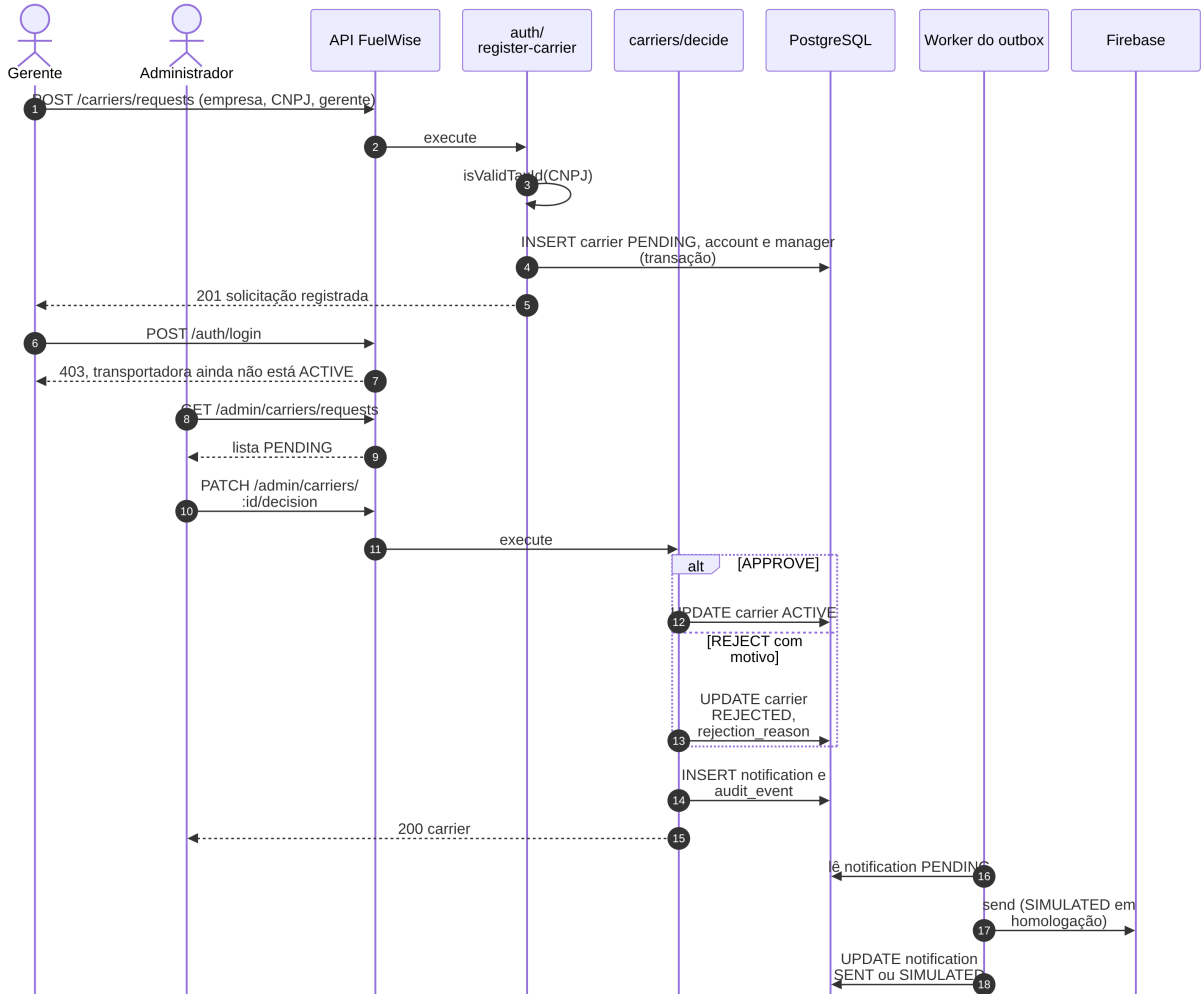


Figura 13: UML de Sequência: aprovação de transportadora (C4 Nível 4)

6.6 Cenário: Registro e Verificação de Denúncia (RF16)

1. Um usuário registra a denúncia contra um posto ou contra um pagamento (nunca os dois).
2. Se o alvo é um pagamento, a API confere se ele pertence à transportadora do usuário e, para caminhoneiros, ao próprio caminhoneiro.
3. A denúncia é gravada como `OPEN`. A restrição `complaint_exclusive_target` no banco garante exatamente um alvo.
4. O administrador modera a denúncia, mudando o status para `UNDER_REVIEW`, `RESOLVED` ou `REJECTED` com uma nota.

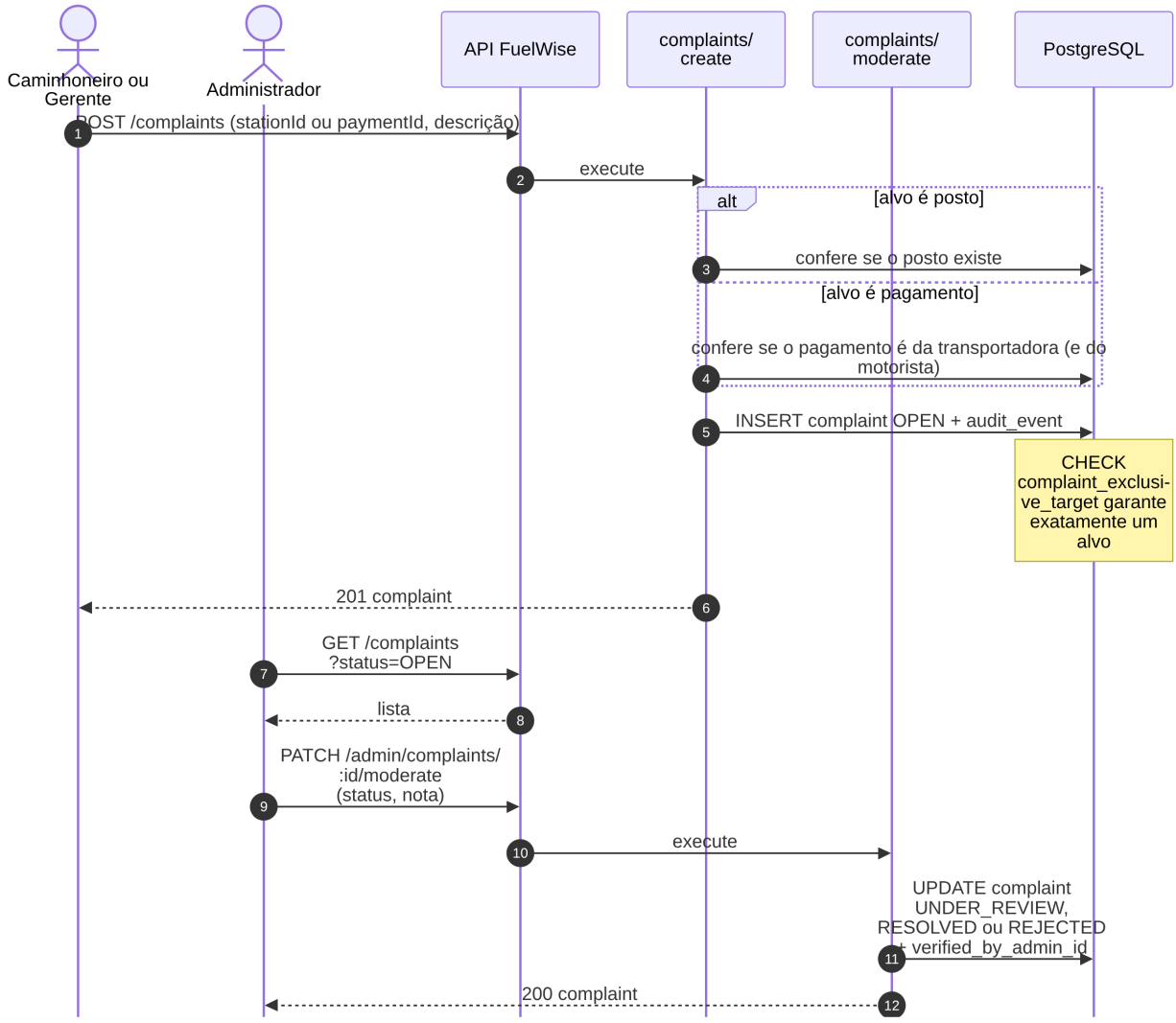


Figura 14: UML de Sequência: denúncia (C4 Nível 4)

6.7 Cenário: Conciliação de Pagamento

Quando o gateway não responde, o pagamento fica `UNKNOWN`. O administrador consulta o status no gateway e, se o resultado já estiver definido, o pagamento é finalizado pelo mesmo `paymentStore.finalize()` usado no fluxo normal. Os passos são:

1. O administrador chama `POST /api/v1/admin/payments/:id/reconcile`.
2. O caso de uso `payments/reconcile` lê o pagamento pela porta `paymentStore.find()`. Se ele já tem resultado definitivo, a regra `assertReconcilable()` do domínio lança conflito, a resposta é 409 e nada muda.
3. Se o pagamento está `PROCESSING` ou `UNKNOWN`, o caso de uso consulta o gateway pela porta `gateway.check()`.
4. Se o gateway ainda não sabe o resultado (`isDefinitive()` falso), a resposta indica que a conciliação continua pendente.
5. Com o resultado `APPROVED` ou `REJECTED`, `paymentStore.finalize()` atualiza o pagamento e o abastecimento e grava auditoria e notificação na mesma transação.

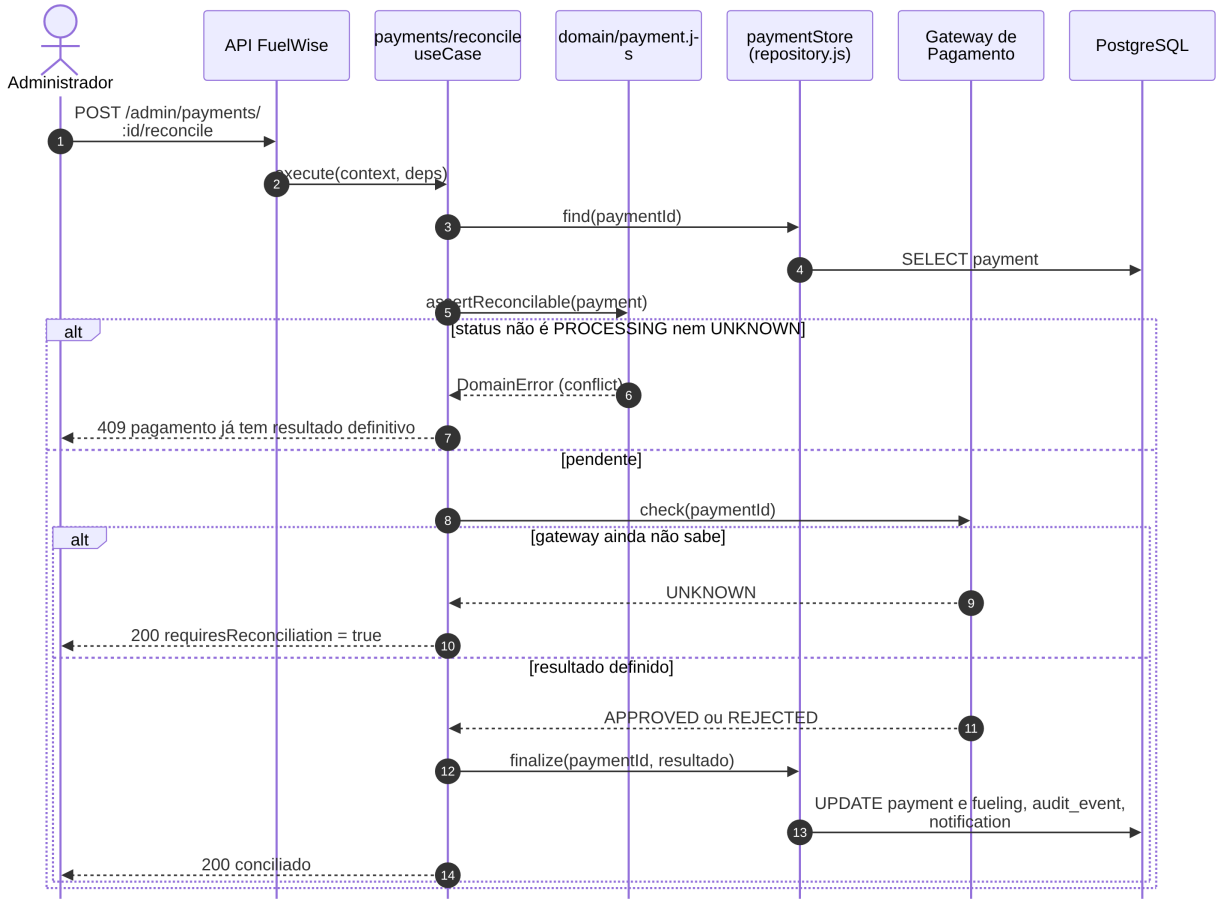


Figura 15: UML de Sequência: conciliação de pagamento (C4 Nível 4)

7. Visão de Implantação

7.1 Nível de Infraestrutura 1: homologação (implementado)

O repositório traz um `compose.yaml` com três contêineres. O banco sobe primeiro; o contêiner `init` aplica as migrações e cria o administrador; só então a API inicia.

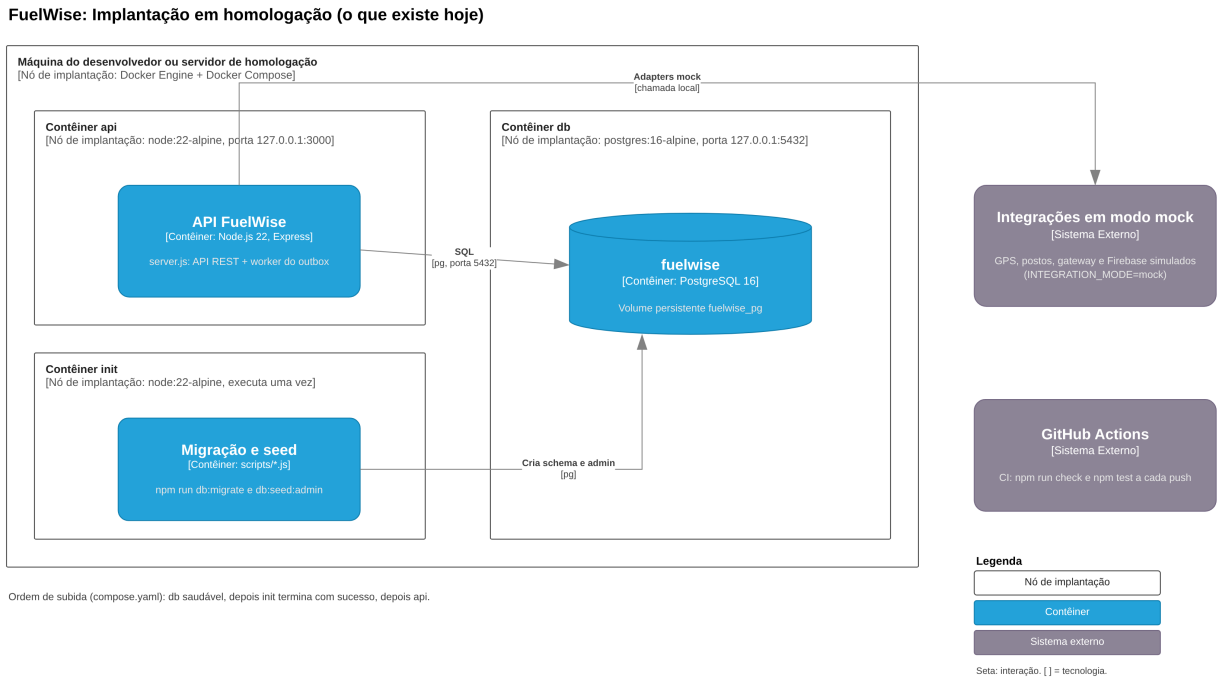


Figura 16: Implantação em homologação

Elemento	Imagem / tecnologia	Observação
`db`	`postgres:16-alpine`	Volume persistente `fuelwise_pg`, porta 5432 apenas em `127.0.0.1`.
`init`	`node:22-alpine`	Executa `npm run db:migrate` e `npm run db:seed:admin` e termina.
`api`	`node:22-alpine`, usuário `node`	API REST e worker do outbox, porta 3000 apenas em `127.0.0.1`.
Integrações	mocks	GPS, postos, gateway e Firebase simulados (`INTEGRATION_MODE=mock`).
CI	GitHub Actions	`npm run check` e `npm test` a cada push e pull request.

7.2 Nível de Infraestrutura 1: produção (proposta no Azure)

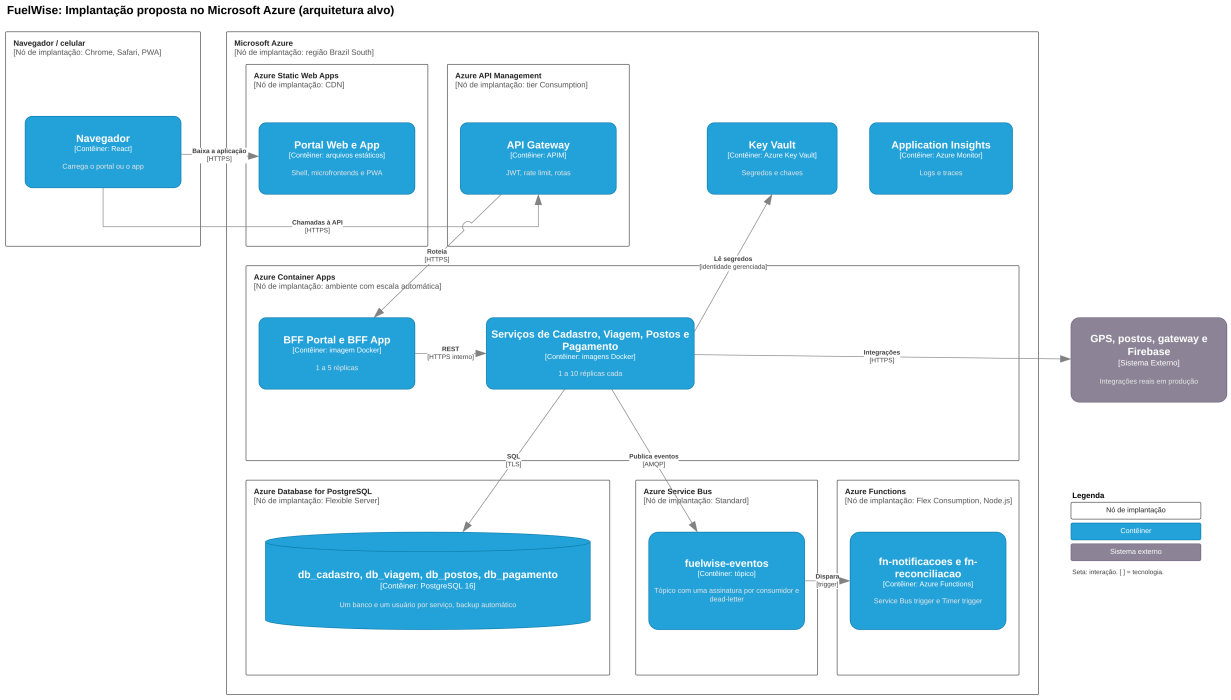


Figura 17: Implantação proposta no Microsoft Azure

Elemento	Serviço Azure	Modelo
Portal Web e App	Azure Static Web Apps	PaaS com CDN
API Gateway	Azure API Management (Consumption)	Serverless
BFFs e serviços	Azure Container Apps (imagens Docker)	PaaS de contêineres com escala automática
Bancos	Azure Database for PostgreSQL, Flexible Server	DBaaS, um banco e um usuário por serviço
Eventos	Azure Service Bus (Standard)	PaaS
Notificações e conciliação	Azure Functions (Flex Consumption)	FaaS
Segredos	Azure Key Vault	PaaS
Observabilidade	Application Insights	PaaS

7.3 Ambientes

Ambiente	Onde	Finalidade
Desenvolvimento	Máquina do desenvolvedor, Docker Compose	Testes locais com integrações mock.
Homologação	Docker Compose ou Azure for Students	Validação funcional; gateway em modo sandbox quando contratado.
Produção	Azure, região Brazil South	Integrações reais. Mocks são bloqueados pelo código (`assertConfig`).

8. Conceitos Transversais

8.1 Segurança

- Autenticação com JWT (HS256, 15 minutos, `issuer` e `audience` verificados) e refresh token rotativo, guardado como hash SHA-256 e revogável.
- Senhas com bcrypt.
- Controle de acesso por perfil (`ADMIN`, `MANAGER`, `DRIVER`) declarado em cada `route.js` e aplicado pelo guard.
- O guard recarrega a conta a cada requisição: conta desativada, perfil diferente do token ou transportadora não ativa resultam em 401.
- `helmet`, CORS restrito e rate limit de 20 requisições a cada 15 minutos nas rotas de login, refresh e cadastro.

8.2 Comunicação

As integrações entre os componentes e serviços externos utilizam APIs REST sobre HTTPS. O cliente HTTP compartilhado (`shared/integrations/httpClient.js`) aplica timeout de 5 segundos e repete com backoff apenas chamadas seguras. Para o gateway, a repetição só é ligada quando o provedor garante idempotência.

8.3 Persistência

O PostgreSQL é o banco principal e a API é a única responsável pelas operações de persistência. Os casos de uso usam SQL explícito pelo `shared/db/pool.js` e abrem transações com `tx()` quando várias tabelas mudam juntas.

Decisões de modelagem do DER (confirmadas no código):

- **Tabela por subtipo de usuário:** `account` guarda os dados comuns de autenticação e cada perfil (`administrator`, `manager`, `driver`) tem a própria tabela, ligada 1:1 a `account`.
- **No máximo um pagamento por abastecimento:** `payment.fueling_id` é único, assim como `payment.idempotency_key`.
- **Denúncia com exatamente um alvo:** a regra é garantida no próprio banco, pela restrição `complaint_exclusive_target`.
- **Histórico de preços:** `fuel_price` guarda `effective_from` em vez de sobrescrever o preço (RF06, RF12).
- **Viagem ativa única:** índices únicos parciais impedem duas viagens `IN_PROGRESS` para o mesmo motorista ou veículo.

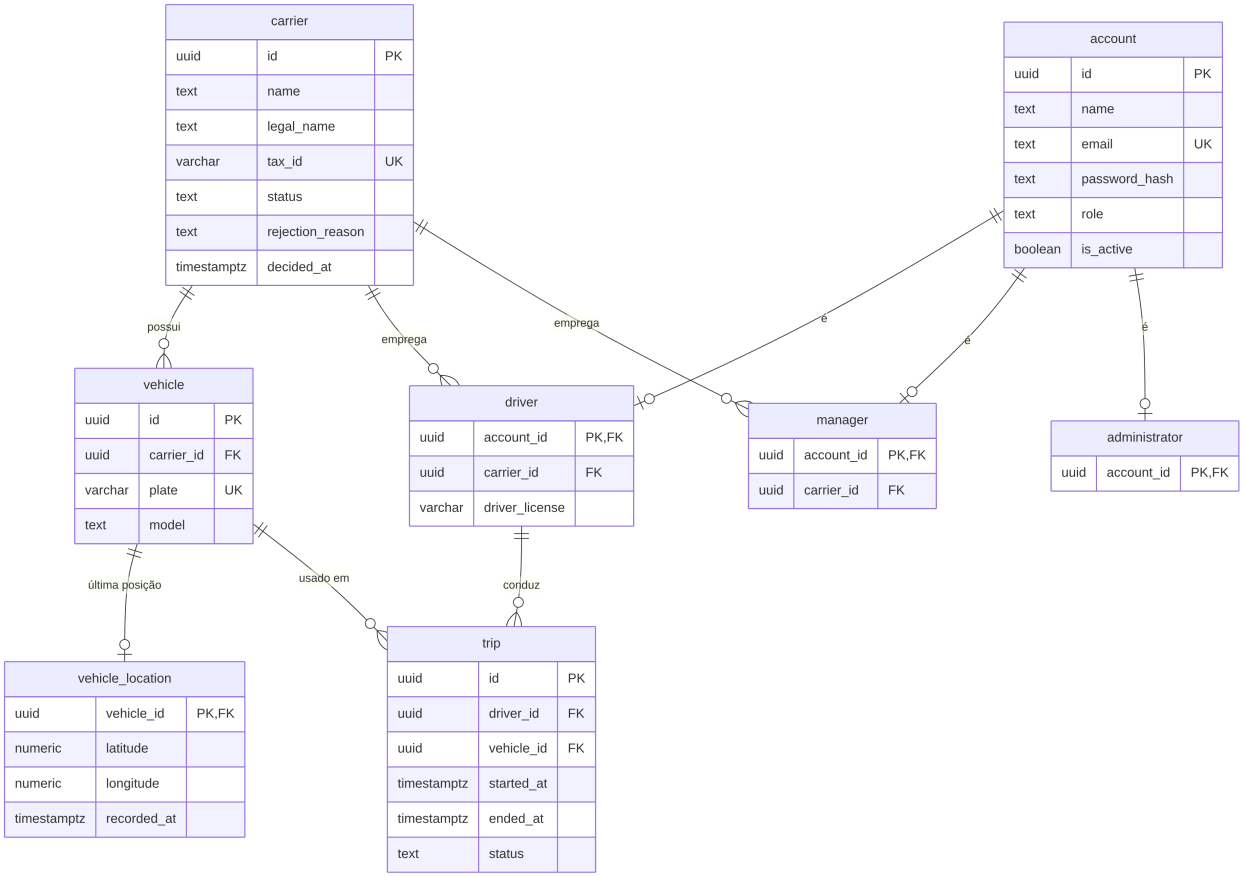


Figura 18: DER: cadastro e frota

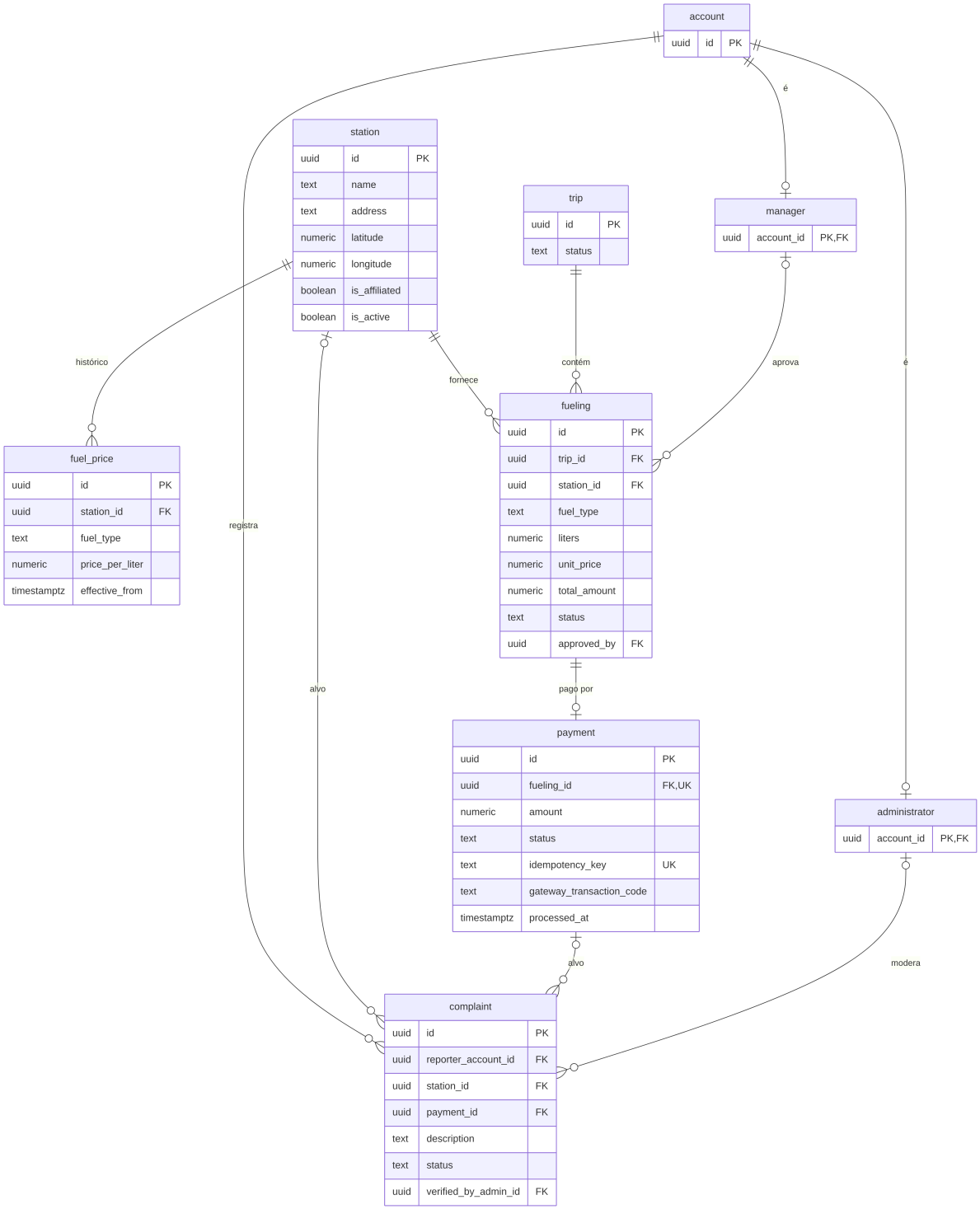


Figura 19: DER: abastecimento, pagamento e denúncia

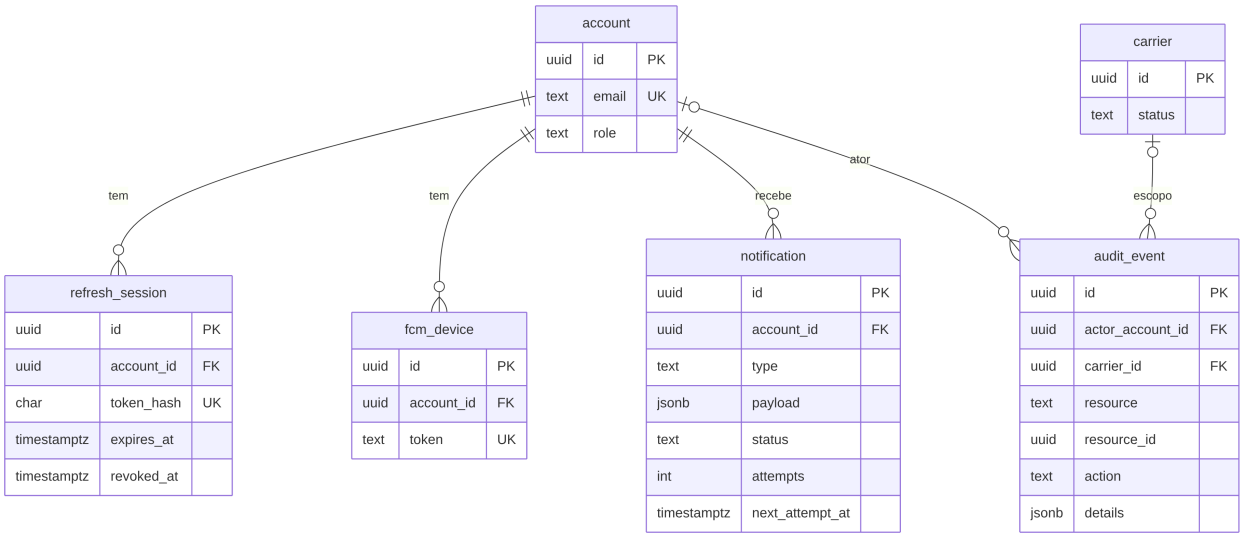


Figura 20: DER: acesso, notificações e auditoria

Tabela	Propósito	Observação de integridade
`account`	Dados comuns de autenticação	E-mail único (sem diferenciar maiúsculas); `role` restrito a três valores.
`administrator`, `manager`, `driver`	Atributos de cada perfil	Chave primária também é chave estrangeira para `account`.
`carrier`	Transportadora cliente	`tax_id` com 14 dígitos e único; `status` `PENDING`, `ACTIVE` ou `REJECTED`.
`vehicle`	Veículo da frota	Placa única.
`trip`	Associação motorista e veículo (RF07)	Uma viagem em andamento por motorista e por veículo.
`vehicle_location`	Última posição conhecida	Usada como fallback quando o GPS falha.
`station`, `fuel_price`	Postos e histórico de preços	Preço sempre positivo; histórico por `effective_from`.
`fueling`	Abastecimento	Valor calculado na API com o preço vigente; status do fluxo de aprovação.
`payment`	Resultado financeiro	`PROCESSING`, `APPROVED`, `REJECTED` ou `UNKNOWN`.
`complaint`	Denúncia (RF16)	Exatamente um alvo: posto ou pagamento.
`notification`	Outbox de notificações	Estados `PENDING`, `PROCESSING`, `SENT`, `SIMULATED`, `FAILED`; até 5 tentativas.
`audit_event`	Auditoria (RF13)	Ator, transportadora, recurso, ação e detalhes.
`refresh_session`, `fcm_device`	Sessões e tokens de dispositivo	Hash do refresh token e token FCM únicos.

8.4 Notificações

As notificações são gravadas na tabela `notification` dentro da mesma transação da operação que as gerou. O worker (`shared/notifications/outbox.js`) roda a cada 10 segundos, bloqueia as pendentes com `FOR UPDATE SKIP LOCKED`, envia pelo Firebase e marca como `SENT`. Em falha, tenta de novo com espera exponencial. Em homologação o envio é simulado e o registro fica como `SIMULATED`, para não dar a falsa impressão de que um push foi entregue.

8.5 Tratamento de Erros e Resiliência

- Timeout de 5 segundos nas integrações externas e repetição com backoff apenas em chamadas idempotentes.

- Falha do GPS: a API devolve a última posição salva com `degraded: true`.
- Falha do gateway durante a cobrança: o pagamento fica `UNKNOWN` e é conciliado depois; nunca é tratado como recusa nem cobrado de novo.
- Regras de negócio violadas lançam `DomainError` (tipos `validation`, `not_found` e `conflict`). O domínio não conhece HTTP: o tratador de erros em `shared/http/errors.js` traduz cada tipo em 400, 404 ou 409.
- Erros do PostgreSQL (duplicidade, referência inexistente, regra violada) são traduzidos em respostas 400 ou 409, sem expor detalhes internos.
- Todas as respostas de erro seguem o formato `{ "error": { "code", "message", "details" } }`.

8.6 Observabilidade

A API registra cada requisição em log estruturado JSON (método, rota, status e duração) e os erros de integração com o identificador do recurso. As operações críticas (pagamento, aprovação de transportadora, abastecimento, viagem e denúncia) ficam também na tabela `audit_event`, com ator, transportadora, recurso e ação. Para o RF13, o endpoint `GET /api/v1/audit/payments` lista os pagamentos com transportadora, motorista, posto e status. Os endpoints `/health/live` e `/health/ready` permitem que o orquestrador verifique a aplicação e o banco.

8.7 Multi-tenancy

Todas as transportadoras usam a mesma API e o mesmo banco. O isolamento é feito nas consultas: o identificador da transportadora vem do usuário autenticado (`actor.carrier_id`), nunca de um parâmetro enviado pelo cliente, e as consultas filtram por ele. Um recurso de outra transportadora responde 404, sem revelar que existe.

8.8 Vertical Slice, SOLID e Clean Architecture

Vertical Slice. O código é dividido pelo que muda junto: cada caso de uso tem a sua pasta com rota e regra. Não existem pastas globais `controllers`, `services` e `repositories`. Um slice não importa outro slice; o que é comum fica em `src/shared`.

SOLID.

Princípio	Onde aparece no código
SRP	`route.js` cuida de HTTP, perfil e validação; `useCase.js` coordena o caso de uso; `repository.js` concentra as transações; `src/domain` guarda só as regras de negócio.
OCP	As fábricas `makeGateway()`, `makeGpsProvider()`, `makeStationProvider()` e `makeNotifier()` devolvem outra implementação sem mudar quem usa.
LSP	Mocks e adaptadores HTTP têm as mesmas assinaturas e o mesmo formato de resposta.
ISP	O pagamento recebe só `reserve/finalize/find` e `charge/check`; o abastecimento, só `request/decide`; o GPS, só `locate`.
DIP	Os casos de uso de `fuelings/create`, `fuelings/approve`, `payments/pay` e `payments/reconcile` recebem `deps.fuelingStore`, `deps.paymentStore` e `deps.gateway` e não importam `pg` nem cliente HTTP. O worker do outbox recebe o `notifier` de `server.js`.

Código 5: `src/app.js` (trecho, quebras de linha ajustadas)

```
export function createApp(overrideDeps={}) {
  const deps={
    gateway:makeGateway(),
    paymentStore:{reserve:reservePayment,finalize:finalizePayment,find:findPayment},
    fuelingStore:{request:requestFueling,decide:saveFuelingDecision},
    gps:makeGpsProvider(),
    stations:makeStationProvider(),
    isPaymentMock:config.paymentProvider==='mock',...overrideDeps
  };
  ...
  [registerSignup, registerLogin, ..., payFueling, getPayment, reconcilePayment, ...]
    .forEach(register => register(app,deps));
```

Clean Architecture. A regra de dependência (o código aponta para dentro) é seguida no núcleo financeiro e nas integrações externas. As regras de abastecimento e pagamento ficam em `src/domain`, que não importa nada de fora (só a biblioteca `decimal.js` para o cálculo em centavos). Os casos de uso do núcleo importam só o domínio; banco e gateway chegam como portas. A correspondência com os anéis vistos em aula é:

Anel	No FuelWise
Entities	`src/domain`: `fueling.js` (valor, aprovação, quando pagar, estado após o pagamento), `payment.js` (idempotência, finalização, conciliação) e `errors.js` (`DomainError`).
Use Cases	`useCase.js` de cada slice e as portas que eles recebem em `deps`.
Interface Adapters	`route.js`, `shared/http`, `repository.js` e os adaptadores de `shared/integrations`.
Frameworks & Drivers	Express, `pg`, PostgreSQL, Firebase, o gateway externo e o composition root (`app.js`, `server.js`).

Código 6: `src/domain/fueling.js` (trecho)

```
export function decide(fueling, decision) {
  if (fueling.status !== FuelingStatus.AWAITING_APPROVAL)
    throw conflict('Abastecimento já analisado');
  if (decision === 'APPROVE') return FuelingStatus.APPROVED;
  if (decision === 'REJECT') return FuelingStatus.REJECTED;
  throw invalid('Decisão deve ser APPROVE ou REJECT');
}

export function statusAfterPayment(paymentStatus) {
  if (paymentStatus === 'APPROVED') return FuelingStatus.PAID;
  if (paymentStatus === 'REJECTED') return FuelingStatus.PAYMENT_REJECTED;
  return FuelingStatus.PAYMENT_UNKNOWN;
}
```

A camada de domínio foi aplicada onde está o risco do sistema: dinheiro e estados do abastecimento. Os demais casos de uso (cadastros, listagens e relatórios) são CRUD e consultas e continuam acessando o banco pelo `shared/db/pool.js` dentro do próprio slice. É uma escolha consciente, seguindo duas orientações das aulas: evitar exagerar a Clean Architecture em CRUD simples e, no Vertical Slice, proteger o núcleo de negócio sem impor a mesma estrutura a todos os slices (ADR-17).

8.9 Testes Unitários de Arquitetura

As regras da Seção 8.8 são verificadas automaticamente pelo arquivo `tests/architecture-rules.test.js`, que lê os imports de todos os arquivos de `src` e falha se uma regra for quebrada. É o mesmo papel que bibliotecas como o

ArchUnit cumprem em Java; aqui o teste foi escrito com o executor nativo do Node.js (`node:test`), sem dependência nova. Ele roda com `npm test` e no GitHub Actions a cada push e pull request.

O repositório já tinha o `tests/architecture.test.js`, que confere regras do modelo de dados e da estrutura. Os dois arquivos juntos cobrem:

Arquivo	Teste	Regra verificada
`architecture-rules.test.js`	Vertical Slice	Um slice não importa outro slice.
`architecture-rules.test.js`	Clean Architecture	`shared` (infraestrutura) não depende de nenhum slice.
`architecture-rules.test.js`	Rotas	`route.js` não acessa banco nem integrações.
`architecture-rules.test.js`	Casos de uso	`useCase.js` e `repository.js` não conhecem Express nem o registro de rotas.
`architecture-rules.test.js`	Domínio (Entities)	Nenhum arquivo de `src/domain` importa outra camada.
`architecture-rules.test.js`	DIP no núcleo financeiro	Os casos de uso de `fuelings/create`, `fuelings/approve`, `payments/pay` e `payments/reconcile` importam só o domínio.
`architecture-rules.test.js`	Composition root	Só `app.js` e `server.js` importam os adaptadores de integração.
`architecture-rules.test.js`	Estrutura	Todo slice tem um `route.js`.
`architecture.test.js`	Integridade do DER	Um pagamento por abastecimento, denúncia com um só alvo e uma viagem ativa por motorista e por veículo.
`architecture.test.js`	Slices de pagamento	`payments/pay` e `payments/reconcile` existem como pastas próprias.

Código 7: `tests/architecture-rules.test.js` (trecho)

```
test('Vertical Slice: um slice nao importa outro slice', () => {
  for (const { file, imports } of inSlices) {
    const own = sliceOf(file);
    for (const target of imports.filter(i => i.startsWith('slices/'))) {
      assert.equal(sliceOf(target), own, `${file} importa ${target}`);
    }
  }
});

test('Clean Architecture: o domínio (Entities) não depende de nenhuma outra camada', () => {
  const domain = graph.filter(g => g.file.startsWith('domain/'));
  for (const { file, imports } of domain) {
    const proibidos = imports.filter(i => !i.startsWith('domain/') && i !== 'decimal.js');
    assert.deepEqual(proibidos, [], `${file} importa ${proibidos.join(', ')}`);
  }
});

test('DIP no núcleo financeiro: casos de uso dependem só do domínio', () => {
  // CORE: fuelings/create, fuelings/approve, payments/pay, payments/reconcile
  for (const slice of CORE) {
    const alvo = graph.find(g => g.file === `slices/${slice}/useCase.js`);
    const proibidos = alvo.imports.filter(i => !i.startsWith('domain/'));
    assert.deepEqual(proibidos, [], `${slice}/useCase.js importa ${proibidos.join(', ')}`);
  }
});
```

Para conferir que os testes realmente pegam violações, foi adicionado de propósito um import de outro slice, um import do Express em um caso de uso e um import do banco em `src/domain` e em `fuelings/create/useCase.js`: os testes correspondentes falharam. Com o código atual, os 38 testes do projeto passam.

Além dos testes de arquitetura, há testes unitários das regras de domínio (`tests/domain.test.js`: valor em centavos, transições de estado, idempotência e conciliação) e testes dos fluxos com portas falsas, sem banco: `tests/payment-flow.test.js` (cobrança única, repetição sem nova cobrança, timeout tratado como `UNKNOWN` e recusa preservada) e `tests/fueling-flow.test.js` (registro, aprovação e conciliação).

8.10 Comunicação por eventos (proposta)

Na arquitetura alvo, a comunicação assíncrona segue o modelo publicação e assinatura: os serviços são os produtores, o tópico `fuelwise-eventos` do Azure Service Bus é o barramento, e cada consumidor tem a sua assinatura. A tabela de outbox continua sendo o ponto de saída: o evento é gravado na mesma transação da operação e publicado depois, como já acontece hoje com as notificações.

Os eventos partem dos pontos do código que hoje gravam uma notificação ou uma auditoria:

Evento	Origem no código atual	Produtor	Consumidor
`CarrierDecided`	`carriers/decide` (notificação `CARRIER_DECISION`)	Serviço de Cadastro	`fn-notificacoes`, avisa o gerente
`ComplaintModerated`	`complaints/moderate` (notificação `COMPLAINT_UPDATED`)	Serviço de Cadastro	`fn-notificacoes`, avisa quem denunciou
`TripStarted`	`trips/start` (notificação `TRIP_STARTED`)	Serviço de Viagem	`fn-notificacoes`, avisa o caminhoneiro
`FuelPriceUpdated`	`stations/add-price` (auditoria)	Serviço de Postos	Nenhum nesta fase; fica disponível para novos assinantes
`FuelingRequested`	`fuelings/create` (notificação `FUELING_PENDING`)	Serviço de Pagamento	`fn-notificacoes`, avisa os gerentes
`FuelingDecided`	`fuelings/approve` (notificação `FUELING_DECISION`)	Serviço de Pagamento	`fn-notificacoes`, avisa o caminhoneiro
`PaymentFinalized`	`payments/pay/repository.js` (notificação `PAYMENT`)	Serviço de Pagamento	`fn-notificacoes`, avisa o caminhoneiro

Mensagens que falharem cinco vezes vão para a fila de *dead-letter*. Como a entrega é “pelo menos uma vez”, os consumidores precisam ignorar mensagens repetidas pelo identificador do evento. Um novo consumidor, por exemplo um relatório em tempo real, entra com uma assinatura nova, sem mudar os produtores.

9. Decisões Arquiteturais

As decisões estão no formato ADR visto em aula: Título, Contexto, Decisão, Status e Consequências. As seis primeiras vêm da Entrega 1 e foram atualizadas; as demais vêm do código e do `docs/DECISIONS.md` do repositório e da arquitetura alvo. O status separa o que já está implementado (*Aceita*) do que é arquitetura alvo (*Proposta*) e do que foi trocado (*Substituída*).

ADR-01: React no frontend

Contexto	Três perfis de usuário usam a mesma aplicação web.
Decisão	React para a interface. Alternativas consideradas: Angular e Vue.js.
Status	Aceita. O frontend ainda não foi implementado.
Consequências	Ecossistema maduro e separação clara entre UI e API. Exige um contrato REST bem definido, documentado em `docs/API.md`.

ADR-02: Java com Spring Boot na API

Contexto	Decisão da Entrega 1.
Decisão	API em Java/Spring Boot.
Status	Substituída pelo ADR-03.
Consequências	Nenhuma no código atual.

ADR-03: Node.js 22, Express, Zod e pg na API

Contexto	Na Semana 10 o grupo reescreveu o backend aplicando Vertical Slice e SOLID. Alternativas: manter Java/Spring Boot, ou usar NestJS ou .NET.
Decisão	Node.js 22 com ESM, Express, validação com Zod e SQL explícito com `pg`.
Status	Aceita.
Consequências	Código enxuto e mesma linguagem das Azure Functions propostas. Sem tipagem estática, os contratos entre portas e adaptadores dependem de testes.

ADR-04: PostgreSQL com transações ACID

Contexto	Pagamentos e auditoria exigem consistência. Alternativas: MySQL e MongoDB.
Decisão	PostgreSQL 16 com restrições no próprio banco (únicos, CHECK, chaves estrangeiras).
Status	Aceita.
Consequências	Regras importantes garantidas mesmo com erro na aplicação. Mudanças de schema exigem migrações controladas.

ADR-05: JWT com refresh token revogável

Contexto	A API precisa escalar horizontalmente sem sessão em memória. Alternativa: sessão com cookie e servidor de sessão.
Decisão	JWT de 15 minutos e refresh token rotativo, guardado como hash e revogável.
Status	Aceita.
Consequências	API sem estado de sessão. A forma de guardar o refresh token no navegador ainda precisa ser definida com o frontend.

ADR-06: Monólito modular

Contexto	Equipe pequena e escopo inicial. Alternativa: microsserviços por domínio desde o início.
Decisão	Uma única API, com módulos separados por domínio de slices.
Status	Aceita para a fase atual. A evolução está no ADR-11.
Consequências	Um deploy só e operação simples. A escala é da API inteira, não por domínio.

ADR-07: Vertical Slice por caso de uso

Contexto	Na organização por camadas, uma mudança em pagamento atravessava controller, service e repository em pastas diferentes.
Decisão	Uma pasta por caso de uso com `route.js` e `useCase.js`, e `repository.js` quando a persistência é complexa. Infraestrutura comum em `src/shared`.
Status	Aceita.
Consequências	Mudanças concentradas em uma pasta. Exige disciplina para não duplicar regras entre slices; a regra “um slice não importa outro” é verificada por teste.

ADR-08: Integrações isoladas em adaptadores injetados

Contexto	GPS, postos, gateway e Firebase não têm fornecedor contratado e precisam de mock em homologação. Alternativa da Entrega 1: chamadas diretas dos serviços às APIs externas.
Decisão	Fábricas em `shared/integrations` devolvem adaptador HTTP ou mock com as mesmas assinaturas; `app.js` injeta nos casos de uso e `server.js` injeta o `notifier` no worker de notificações. Mocks são proibidos em produção.
Status	Aceita.
Consequências	Troca de fornecedor sem alterar regras de negócio. Um teste de arquitetura garante que só `app.js` e `server.js` importam os adaptadores.

ADR-09: Pagamento reservado antes da cobrança

Contexto	Uma falha de rede no meio da cobrança pode deixar o gateway e o banco divergentes.
Decisão	O gerente aprova o abastecimento antes; o pagamento é gravado `PROCESSING` antes da chamada; chave de idempotência por abastecimento; sem resposta, o status é `UNKNOWN` e o administrador concilia.
Status	Aceita.
Consequências	Nenhuma cobrança duplicada e nenhuma recusa presumida. A conciliação ainda é manual (ver ADR-15).

ADR-10: Outbox de notificações no PostgreSQL

Contexto	Enviar o push dentro da requisição deixaria a operação lenta e poderia perder a notificação se o Firebase falhasse.
Decisão	Gravar a notificação na mesma transação e enviar por um worker com repetição.
Status	Aceita.
Consequências	Nenhuma notificação perdida por falha externa. O envio tem atraso de alguns segundos.

ADR-11: Microsserviços com banco por serviço

Contexto	Com mais transportadoras, pagamento e viagens terão carga e ritmo de mudança diferentes do cadastro.
Decisão	Extrair quatro serviços (Cadastro, Viagem, Postos, Pagamento) a partir dos domínios de slices, cada um com o seu banco PostgreSQL.
Status	Proposta.
Consequências	Deploy e escala independentes. Sem JOIN entre serviços; os dados de outro serviço chegam por eventos ou por API, com consistência eventual.

ADR-12: API Gateway e BFF

Contexto	O portal do gerente e do administrador precisa de telas com dados agregados; o app do caminhoneiro precisa de respostas pequenas.
Decisão	Azure API Management como entrada única e dois BFFs: BFF Portal e BFF App.
Status	Proposta.
Consequências	Segurança centralizada e respostas adequadas a cada cliente. Mais dois serviços para manter; BFF não deve ter regra de negócio.

ADR-13: Microfrontend no Portal Web

Contexto	As telas de frota, financeiro e administração evoluem em ritmos diferentes.
Decisão	Portal com um shell e três microfrontends (Module Federation). O app do caminhoneiro continua como uma aplicação única.
Status	Proposta.
Consequências	Publicação independente de cada área. Exige uma biblioteca de componentes comum para manter o visual consistente.

ADR-14: Eventos no Azure Service Bus

Contexto	Depois de um pagamento ou de uma decisão, outros módulos precisam reagir sem chamadas em cadeia.
Decisão	Publicar eventos de domínio no tópico `fuelwise-eventos`, usando o outbox atual como publicador.
Status	Proposta.
Consequências	Módulos desacoplados e mensagens guardadas se um consumidor cair. Depuração mais difícil, que exige rastreamento distribuído.

ADR-15: Azure Functions para notificações e conciliação

Contexto	O envio de notificações e a conciliação rodam por evento ou por horário e ficam parados a maior parte do tempo.
Decisão	`fn-notificacoes` (gatilho do Service Bus) e `fn-reconciliacao` (gatilho de horário, diário) em Node.js.
Status	Proposta.
Consequências	Custo por uso e conciliação automática. Dependência do Azure e possível *cold start*.

ADR-16: Testes de arquitetura no CI

Contexto	Regras de arquitetura descritas só no documento tendem a ser quebradas com o tempo.
Decisão	`tests/architecture-rules.test.js` verifica as regras de dependência e roda no GitHub Actions.
Status	Aceita.
Consequências	O documento e o código continuam alinhados. Mudanças de estrutura exigem atualizar as regras junto.

ADR-17: Camada de domínio no núcleo financeiro

Contexto	As regras de valor, aprovação e pagamento estavam espalhadas entre casos de uso e consultas SQL, e só podiam ser testadas com o banco. Alternativas: manter os casos de uso acessando o banco direto, ou aplicar camadas de domínio e repositório em todos os 34 slices.
Decisão	Criar `src/domain` com as regras de abastecimento e pagamento e fazer os casos de uso de `fuelings/create`, `fuelings/approve`, `payments/pay` e `payments/reconcile` dependerem só do domínio e das portas `fuelingStore`, `paymentStore` e `gateway`. Os slices de CRUD e consulta continuam com acesso direto ao banco.
Status	Aceita.
Consequências	As regras de dinheiro e de estado ficam num lugar só e são testadas sem banco (`domain.test.js` e `fueling-flow.test.js`). Os testes de arquitetura impedem que o domínio passe a depender de outra camada. Os demais slices não ganham essa proteção, o que é aceitável porque não têm regras de negócio relevantes.

10. Requisitos de Qualidade

10.1 Árvore de Qualidade

- Qualidade
  - Segurança
    - Autenticação dos usuários (Q1)
    - Controle de acesso por perfil (Q2)
    - Isolamento entre transportadoras (Q3)
  - Confiabilidade
    - Pagamento sem cobrança duplicada (Q4)
    - Pagamento sem resultado presumido (Q5)
    - Registro de auditoria e de denúncias (Q6)
  - Disponibilidade
    - Localização com fallback (Q7)
    - Disponibilidade da API (Q8)
  - Desempenho
    - Consultas de postos (Q9)
  - Manutenibilidade
    - Troca de fornecedor de integração (Q10)
    - Regras de arquitetura verificadas (Q11)

10.2 Cenários de Qualidade

Id	Qualidade	Cenário	Métrica
Q1	Segurança	Um usuário tenta usar um token expirado ou de conta desativada.	100% das tentativas recebem 401.
Q2	Segurança	Um caminhoneiro chama uma rota de gerente ou administrador.	100% das rotas protegidas por perfil respondem 403.
Q3	Segurança	Um gerente pede um veículo de outra transportadora.	Resposta 404, sem revelar que o veículo existe.
Q4	Confiabilidade	O caminhoneiro aperta “pagar” duas vezes.	Uma única cobrança enviada ao gateway (coberto por `payment-flow.test.js`).
Q5	Confiabilidade	O gateway não responde durante a cobrança.	Pagamento `UNKNOWN`, nunca `REJECTED` presumido (coberto por teste).
Q6	Confiabilidade	Um pagamento é finalizado.	100% das finalizações com registro em `audit_event`.
Q7	Disponibilidade	O serviço GPS fica fora do ar.	A API devolve a última posição salva com `degraded: true`.
Q8	Disponibilidade	Operação normal durante um mês.	API disponível em pelo menos 99% do tempo (ainda não medido).
Q9	Desempenho	Busca de postos num raio de 20 km.	Resposta em até 2 s em 95% das requisições (ainda não medido).
Q10	Manutenibilidade	A transportadora troca de gateway de pagamento.	Mudança só em `shared/integrations/paymentGateway.js`.
Q11	Manutenibilidade	Um desenvolvedor importa outro slice ou o Express num caso de uso.	O teste de arquitetura falha no CI antes do merge.

11. Riscos e Débitos Técnicos

Risco ou débito	Impacto	Mitigação
Dependência de serviços externos sem fornecedor contratado	Os adaptadores HTTP podem não bater com a API real de cada fornecedor.	Contratos isolados em `shared/integrations`; ajustar só o adaptador na contratação.
Conciliação de pagamentos manual	Pagamentos `UNKNOWN` dependem de o administrador agir.	Conciliação automática diária (ADR-15).
Clean Architecture só no núcleo financeiro	Os slices de CRUD e consulta acessam o banco direto e só são testados com PostgreSQL. Se ganharem regras de negócio, essas regras podem ficar no SQL.	Quando um slice ganhar regra de negócio, mover a regra para `src/domain` e incluir o slice na lista de núcleo verificada pelo teste de arquitetura (ADR-17).
Sem testes de ponta a ponta	Testes atuais são unitários e de arquitetura; integração com PostgreSQL e parceiros reais não foi executada.	Testes de integração com o banco do Docker Compose no CI.
Requisitos de desempenho e disponibilidade não medidos	Q8 e Q9 são metas, sem medição.	Teste de carga antes da produção.
Frontend não implementado	Os fluxos ainda não foram validados com usuários.	Implementar o portal e o app consumindo `docs/API.md`.
Escalabilidade do monólito	Crescimento de transportadoras exige escalar a API inteira.	Evolução para os serviços da Seção 4.2 quando houver carga.
Denúncias sem prazo de resposta	RF16 tem fluxo de moderação, mas sem SLA nem regra de suspensão de posto.	Definir SLA e critérios de suspensão na próxima iteração.

12. Glossário

Termo	Definição
FuelWise	Plataforma para controle de combustível, pagamentos e gestão da frota.
Transportadora (`carrier`)	Empresa cliente que usa o FuelWise para administrar a frota e os abastecimentos.
Caminhoneiro (`driver`)	Usuário que realiza viagens e abastecimentos.
Gerente (`manager`)	Usuário que administra a transportadora, a frota e aprova os abastecimentos.
Administrador da Plataforma	Usuário que aprova transportadoras, gerencia postos, concilia pagamentos e trata denúncias.
Posto (`station`)	Estabelecimento de combustível cadastrado ou consultado na plataforma.
Abastecimento (`fueling`)	Compra de combustível registrada durante uma viagem.
Viagem (`trip`)	Associação entre um caminhoneiro e um veículo durante uma operação.
Denúncia (`complaint`)	Registro de irregularidade em um posto ou em um pagamento, verificado por um administrador.
Gateway de Pagamento	Serviço externo que processa os pagamentos.
Conciliação	Consulta ao gateway para definir o resultado de um pagamento `UNKNOWN`.
Idempotência	Garantia de que repetir a mesma requisição não gera uma nova cobrança.
JWT	JSON Web Token, usado para autenticação e controle de acesso.
Mock	Implementação simulada de uma integração, usada em homologação.
Outbox	Tabela onde a notificação é gravada na mesma transação da operação, para envio posterior.
Domínio	Camada `src/domain` com as regras de negócio do abastecimento e do pagamento, sem dependência de banco, HTTP ou framework (anel Entities da Clean Architecture).
`DomainError`	Erro lançado por uma regra de domínio (`validation`, `not_found` ou `conflict`); a camada HTTP converte em 400, 404 ou 409.
Porta	Contrato (conjunto de métodos) que um caso de uso recebe por injeção.
Adaptador	Implementação concreta de uma porta, por exemplo o cliente HTTP do gateway.
Slice	Pasta de um caso de uso no padrão Vertical Slice, com `route.js` e `useCase.js`.
Composition root	Ponto onde as dependências são criadas e injetadas (`src/app.js` e `src/server.js`).
Clean Architecture	Organização em que o código depende das regras de negócio, e não de banco, framework ou serviço externo.
Teste de arquitetura	Teste automatizado que falha quando o código quebra uma regra de dependência ou de estrutura.
BFF	Backend for Frontend: backend específico para um tipo de cliente.
API Gateway	Ponto único de entrada das requisições externas.
Microsserviço	Serviço pequeno, com deploy próprio, responsável por um domínio do negócio.
Database per Service	Cada microsserviço tem o seu banco e nenhum outro serviço acessa esse banco diretamente.
Microfrontend	Parte da interface com código e publicação próprios, montada dentro de um shell.
EDA	Arquitetura orientada a eventos: produtores publicam eventos num barramento e consumidores reagem a eles.
Serverless	Execução de funções sob demanda, sem gerenciar servidores, com cobrança por uso (Azure Functions).

Repositório do código

O código descrito neste documento está no GitHub:

Item	Valor
Repositório	https://github.com/LucasTheiss/fuelwise-architecture
Branch	`vertical-slice`
Casos de uso (Vertical Slices)	`src/slices/<domínio>/<ação>/`
Regras de domínio (Entities)	`src/domain/`
Infraestrutura compartilhada	`src/shared/`
Composition root	`src/app.js` e `src/server.js`
Testes, incluindo os de arquitetura	`tests/`
Esquema do banco	`database/migrations/001_initial.sql`
Documentação técnica	`README.md` e `docs/`
Integração contínua	`.github/workflows/ci.yml` (GitHub Actions)

Os trechos de código deste documento foram copiados desse repositório, e o arquivo de origem aparece no título de cada trecho. O GitHub Actions executa `npm run check` e `npm test` a cada push e pull request; no último commit da branch `vertical-slice`, os 38 testes passaram.

Para subir o ambiente de homologação e rodar os testes:

Código 8: Comandos para subir o ambiente e rodar os testes

```
cp .env.example .env      # trocar JWT_SECRET e ADMIN_PASSWORD
docker compose up -d --build
npm install
npm test
```

Depois de subir o ambiente:

- API: `http://localhost:3000/api/v1`
- Verificação de saúde: `http://localhost:3000/health/ready`